Automating Induction-Reduction for Concurrent Object Proofs

MIHAI NICOLA, Stevens Institute of Technology, USA
CONSTANTIN ENEA, Ecole Polytechnique, LIX, France
ERIC KOSKINEN, Stevens Institute of Technology, USA

A principal challenge in reasoning about concurrent objects, is to develop an intuitive correctness argument that nonetheless applies to unboundedly many threads. Two key strategies for tackling this unbounded domain are induction (e.g. over a trace or over threads) and Lipton's reduction (re-ordering a trace based on commutativity) and recent works are increasingly incorporating both strategies.

In this paper we integrate induction over traces with unbounded left/right reductions at each step, and show how to reduce this combination to standard building blocks: Hoare logic and contextual commutativity, along with a few automaton operations. The upshot is that concurrent object proofs can be organized around a few user-provided scenarios written as "induction-reduction" specifications or "IRSpecs" (which extend to unbounded thread interleaving) and, through new proof rules, they can be automatically verified. We implemented a simple prototype tool and show that IRSpecs reduce reasoning about difficult concurrent objects (e.g. the Michael-Scott Queue, the Hendler *et al.* Elimination Stack, the Harris *et al.* RDCSS) to a few intuitive canonical executions.

ACM Reference Format:
Mihai Nicola, Constantin Enea, and Eric Koskinen. 2026. Automating Induction-Reduction for Concurrent Object Proofs. 1, 1 (April 2026), 47 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Lock-free concurrent implementations of standard abstract data types, often called "concurrent objects" [26], leverage efficient hardware primitives for performance. However, identifying bugs within these implementations continues to be challenging, and formally verifying their correctness (typically via linearizability [26]) requires substantial effort. To formally verify concurrent objects program logics have been developed including Owicki-Gries [45], Rely/Guarantee [27], Concurrent separation logic [4, 44], RGSep [53], Deny-Guarantee [12], Views [11], Iris [28, 30] as well as interactive proof tools for such logics.

Such program logics typically focus on thread-local reasoning, wherein the effects of threads are captured in state invariants. Other approaches [15, 24] instead focus on re-ordering executions such as through Lipton's reduction [39] in order to reduce interleaved executions to executions that are more sequential. For finite threads, there are exponentially many interleavings, and so strategies such as partial-order reduction [21] have been applied to tame the problem. Such strategies use commutativity arguments to reduce them problem to fewer traces (e.g., Mazurkiewicz traces [40]).

Authors' Contact Information: Mihai Nicola, Stevens Institute of Technology, Hoboken, USA, lnicola@stevens.edu; Constantin Enea, Ecole Polytechnique, LIX, Paris, France, cenea@lix.polytechnique.fr; Eric Koskinen, Stevens Institute of Technology, Hoboken, USA, eric.koskinen@stevens.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/4-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

In the context of unboundedly many threads, the problem goes beyond exponential to infinitely many interleavings, making the set of all executions seem downright unapproachable. Increasingly recent works have sought to *combine* induction with Lipton's reduction to tackle this problem (e.g. inductive sequentialization [32]). One more recent approach [16] introduced a small context-free "quotient language" for describing the set of unbounded interleaved executions by, for example, lining up the actions of any n threads into n-specific canonical forms. The upshot is that the executions of concurrent objects can be described as a collection of "scenarios" that more closely resemble the intuitive, operational correctness arguments given by algorithm designers. The work however does not yet provide a clear method of combining induction with reduction to obtain those quotients, nor does it provide a clear path to automation, thus leaving users to write *ad hoc* proofs.

In this paper we introduce a new route toward automation of induction and reduction for unbounded thread reasoning, integrating induction over execution traces with, at each induction step, unbounded reductions specific to that step. We proceed in two steps: (i) introducing a way to *express* the induction-reduction combination, and then (ii) introducing proof rules that reduce their soundness to automation-friendly building blocks.

In the first step, these induction-reduction expressions or "IRSpecs" are written as concise tuples of the form $(a, R \triangleright N \triangleleft L)$ specifying an induction action a (that many threads may take in an execution) and, for each kth instance a_k of a in an execution, how to reduce a portion of the execution by moving some other actions R_k (instance of R at this step k) to the right and actions L_k to the left to be alongside a non-moving "anchor" N_k that contains a_k. Naturally, in the subsequent induction step for the $(k+1)$th instance of a other actions will be moved to the right/left. The conditions under which these reorderings are sound are defined via a set of proof rules that additionally allow to compose IRSpecs. Some concrete and simple examples will be provided in Section 2.

The benefit of IRSpecs and our proof rules is that they simplify the complexity of concurrent object reasoning that is due to thread interleaving. IRSpecs isolate important steps (e.g. a successful CAS) and the interaction with those steps are organized into canonical execution fragments via commutative reorderings. This makes reasoning about properties such as linearizability simple because the linearization points are directly available (e.g. the successful CAS at b). In Sec. 8 we show that, across several canonical objects, that IRSpecs reduce reasoning to a handful of intuitive cases (or "scenarios"), which is much simpler than reasoning about arbitrary thread interleaving.

Challenges and Contributions. We show how induction-refinement can be reduced to automation-friendly primitives, addressing the following challenges:

1. *Induction-Reduction.* (Sec. 5) We first formally define the *semantics* of induction-reduction as tuples of the form $(\mathfrak{a}, \mathrm{atom/coop}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L}))$ where $\mathfrak{a}$ is an inductive action that can occur multiple times in an execution and, at each inductive step, unboundedly many threads' actions R can be moved to the right and actions L can be moved to the left, to be grouped around that step's anchor N. The semantics of an atom IRSpec groups unboundedly many instances of an individual threads' actions, whereas a coop IRSpec groups unboundedly many other threads around the corresponding instances from the associated atom IRSpec. Next, a collection of IRSpecs can be *composed* together in a calculus that applies a series of IRSpec refinement steps, taking into account that some steps are to be taken in a specific order (such as an atom step occurring before its corresponding coop step). Finally, we define *orthogonality* of IRSpecs to prevent them from "competing" with each other, and *coverage* to ensure enough IRSpecs have been provided to cover all paths of the object's implementation.

2. *Proof rules.* (Sec. 6) While the semantics of IRSpecs seem appealing, the next question is whether an IRSpec can be validated automatically given that it involves quantification over unboundedly

many threads' behavior. We thus introduce proof rules for atom and coop IRSpecs, reducing induction-reduction to automation friendly primitives: Hoare logic, contextual commutativity, and syntactic conditions reducible to finite-state automata operations. In addition, the proof rules can also be used to *derive* a canonical description of the set of all interleavings of an object that in general, over-approximates a sound reduction.

3. *Implementation and user-interactivity.* (Sec. 7) We provide a prototype tool QRef, written in OCaml that takes an input concurrent object implementation and user-provided IRSpecs, and then automatically validates the syntactic conditions. These checks allow for some interactivity: as a user starts to write candidate IRSpecs QRef will report whether they are incorrect, e.g., pairs of them are not orthogonal. If the user has not provided enough IRSpecs, QRef will provide a path through the implementation that has not yet been covered (which it obtains from a failed coverage check). QRef constructs the necessary automata in OCaml, and uses `libvata` [36] to perform automaton operations/queries, and CVC5 [2] to generate counterexample paths.

4. *Evaluation.* (Sec. 8) We finally provide an evaluation showing that IRSpecs reduce reasoning to a canonical representation for a variety of notorious concurrent objects (including the Michael-Scott queue [41], Treiber's Stack [49], the Hendler *et al.* Elimination Stack [25], and the Harris *et al.* Restricted Double-Compare, Single-Swap (RDCSS) [23]). We discuss user-interactivity on the MSQ and report that QRef can validate the syntactic conditions of those objects' IRSpecs in a matter of seconds.

Limitations. QRef implements the key checks for orthogonality, some notion of bijection described later, and coverage which are new aspects of the theory and assumes that users provide left/right movers that commute up to the post/pre (resp.) contexts of each anchor. Checking those commutativity conditions can be accomplished with known techniques/tools (e.g. [7, 8]). We thus leave integration with those tools to future work and instead manually provided the necessary contexts for the case studies (see Apx. D of the supplement).

Supplement. Included with the supplement is the source code of our prototype tool QRef, which we plan to submit for artifact evaluation and publicly release.

2 Overview

We consider a concurrent Counter to illustrate (i) how IRSpecs characterize unbounded thread interleavings concisely in a few simple expressions, and (ii) how the proof rules reduce reasoning about IRSpecs to automation-friendly primitives. We also summarize QRef and how it can guide a user in writing IRSpecs. The methods for incrementing and decrementing are as follows:

```
int increment() {
 while (true) {
   int c = ctr;
   if (cas(ctr,c,c+1)) return c;
  } }
```

```
int decrement() {
 while (true) {
  int c = ctr;
  if (cas(ctr,c,c-1))  return c;
 } }
```

Both methods return the previous value of the counter. Each method consists of a retry-loop that reads shared variable `ctr` representing the counter and attempts to update it using a Compare-And-Swap (CAS). A CAS atomically tests whether `ctr` equals the second argument and if so, then it assigns the value specified by the third argument. If the test fails, then the CAS has no effect. The return value of CAS represents the truth value of the equality test. If the CAS is unsuccessful, *i.e.* it returns *false*, then the method retries the same steps in another iteration.

These methods can be expressed as simple Kleene Algebra with Test (KAT) expressions, where there are symbols for each atomic statement, and the CAS operation is split into two cases depending

on success. The expressions are:

$$\left(\texttt{c:=ctr}_R\ (\texttt{cas(ctr,c,c+1)/true}_I\ \texttt{return c}_X\ +\ \texttt{cas(ctr,c,c+1)/false}_F)\right)^*$$
$$\left(\texttt{c:=ctr}_S\ (\texttt{cas(ctr,c,c-1)/true}_D\ \texttt{return c}_Y\ +\ \texttt{cas(ctr,c,c-1)/false}_G)\right)^*$$

Above we have given shorthand names for each symbol such as R for `c:=ctr`. Note that we intentionally distinguish between identical statements occurring in different program locations (e.g. R versus S).

The executions of the concurrent counter are interleavings of an arbitrary number of increment or decrement invocations from an arbitrary number of threads. Nevertheless, this can be reduced to a far simpler canonical form through four IRSpecs, which we will define below. Each IRSpec $(\mathfrak{a}, \mathrm{atom}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L}))$ denotes *an induction along with a reduction to be applied at each inductive step*:

- (i) $\mathfrak{a}$ is an induction action that could be taken by unboundedly many threads in an execution, over which induction will take place, and
- (ii) for *each* step k of the induction (i.e. kth occurrence of $\mathfrak{a}$),
  - N: a KAT expression of actions that will remain unmoved (an "anchor"),
  - R: a KAT expression of actions that can move to the right up to N, and
  - L: a KAT expression of actions that can move to the left down to N.

Thus the IRSpec $(\mathfrak{a}, \mathrm{atom}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L}))$ can be thought of as an *operator on sets of traces* that produces a smaller set of traces (i.e. a reduction). In the case of the Counter, we have four such IRSpecs:

$$\begin{aligned}
\alpha_{incS} &\equiv (I, \mathrm{atom}(R \triangleright I \triangleleft X)),\\
\alpha_{decS} &\equiv (D, \mathrm{atom}(S \triangleright D \triangleleft Y)),\\
\alpha_{incF} &\equiv (\{I, D\}, \mathrm{coop}(R \triangleright (RIX + SDY) \triangleleft F)), \text{and}\\
\alpha_{decF} &\equiv (\{I, D\}, \mathrm{coop}(S \triangleright (RIX + SDY) \triangleleft G)).
\end{aligned}$$

It turns out that *any* arbitrary interleaved execution ε of unboundedly many threads concurrently acting as the Counter can be reduced to a simple canonical form by using these four IRSpecs. The first IRSpec α_{incS} says that, inducting over each occurrence k of an increment's successful CAS $\texttt{cas(ctr,c,c+1)/true}_I$, we can move a read R corresponding of the kth instance of I_k to the right up to that anchor I_k, (and similarly move a return action X to the left up to that I_k). After reducing with α_{incS}, all successful increment read-CAS-return iterations will be re-organized and grouped together in execution ε. Next, α_{decS} specifies a second induction, and at each step of the induction a successful decrement read-CAS-return will be grouped per a kth instance $S_k \triangleright D_k \triangleleft Y_k$. These reductions so far are called "atom" because they form atomic sections.

What remains is to organize the iterations (from either an increment or a decrement) that *fail* a CAS due to another thread's successful CAS which has now been grouped as either RIX or SDY. The "coop" IRSpec α_{incF} now specifies how to group incrementing threads' iterations that performed a read R before the successful RIX or SDY and then failed as F. Specifically, we now induct again over the occurrences of I or D from the previously-grouped successful iterations RIX or SDY and, for each kth induction step now move the actions of the threads that failed due to that I_k or D_k: i.e., we move the reads R to the right up to the next anchor $(RIX)_k$ or $(SDY)_k$ and move the CAS failures F to the left up to $(RIX)_k$ or $(SDY)_k$. (Below we will discuss a bijection condition ensuring that this is possible.) Finally, we similarly use α_{decF}, inducting again over previously grouped RDX/SDY instances, moving reads S to the right and failures G to the left up to each $(RDX)_k$ or $(SDY)_k$ instance. These reductions are called "coop" referring to the fact that they group actions from other threads around the actions of a different (perhaps not-so-"cooperative") thread. These four IRSpecs, when composed together, (along with a simple sequencing rule) reduce every

execution's unbounded arbitrary thread interleaving to the format:

$$[(R+S)^*(RIX+SDY)(F+G)^*]^*. \tag{1}$$

Thanks to inductively-applied commutativity, this expression is a sound representation of unboundedly many threads operating on the original object (i.e., over-approximates a reduction of all interleavings).

2.1 From induction-reduction to automation

The crux of this paper are proof rules that reduce the above combination of induction & reduction to primitives that are amenable to verification. Roughly, the proof rules have the shape:

$$\textsc{Atom}\ \frac{\{B_i\}N_i\{B'_i\} \quad R_i \triangleright B_i \quad B'_i \triangleleft L_i \quad \ldots}{T \supseteq lfp_{\mathfrak{a}_i}[R_i \cdot N_i \cdot L_i](T)} \qquad \textsc{Coop}\ \frac{\{C_j\}N_i\{C'_j\} \quad R_j \triangleright C_j \quad C'_j \triangleleft L_j \quad C_j = sp_j(R) \quad C'_j = wp_j(L) \quad \ldots}{T \supseteq lfp_{\mathfrak{a}^{\cup}_j}[R_j \cdot N_i \cdot L_j](T)}$$

Note that these rule sketches are for intuition and we provide the formal rules (five in total) and proofs in Sec. 6. The Atom rule reduces the process of forming atomic paths out of successful "happy paths" to Hoare logic and contextual commutativity (along with some syntactic automata conditions discussed below). When this rule is instantiated for α_{incS}, it rewrites traces up to the least fixpoint, taking each occurrence i of successful CAS action $\mathfrak{a}_i$, grouping $R_iN_iL_i$ as an atomic section, thus reducing interleavings, yet maintaining trace-refinement. In each occurrence, there is a context of $\{B_i\}N_i\{B'_i\}$ (i.e. a Hoare triple), and commutativity conditions to move the read operations (specified by R) to the right up to context B_i, and the return statement (specified by L) to the left down to context B'_i. Note that these are "big-step" movers (see also [19]) in the sense that an action can move to the right/left across multiple actions *even if* they don't pair-wise commute with those actions, as long as ultimately they commute with the sum of those actions, captured by the context B_i/B'_i.

The second Coop rule works in tandem with Atom, and lets us further group the other threads (such as those that fail their CAS in α_{incF}) alongside the previously-grouped successful atomic sections' induction actions in the set $\mathfrak{a}^{\cup}_j$ (for the Counter $\mathfrak{a}^{\cup} = \{I, D\}$). This rule again reduces the problem to Hoare logic, contextual commutativity, strongest-pre/weakest-post, stability under other Atoms' actions, and syntactic automaton conditions.

To use these rules, there are three remaining conditions and we show that they are all amenable to syntactic automaton reasoning. First, each IRSpec must be well-formed in the sense that there is a *bijection* between the induction actions $\mathfrak{a}$ and the anchor N. The inductive actions does not need to be exactly N but rather, intuitively, whenever one inductive action occurs, there must inevitably be a corresponding anchor on the same path of the implementation (and vice versa) even if it occurs earlier or later. Second, IRSpecs should not "compete" with each other so they must be *orthogonal*: a prefix of on IRSpec should not be a suffix of another or vice-versa. Nor should one be a KAT-factor of another. Finally, a collection of IRSpecs should have enough *coverage* so that all of the object's behaviors are captured.

We show that all of these conditions can be captured with automaton operations, which are also automatically checked by our tool discussed next. If a collection of IRSpecs satisfy all of these conditions, then they can be *composed* together. This composition of the IRSpecs *constructs* the canonical representation in (1), which per Theorem 6.4, represents the set of all unbounded interleaved executions of all increment/decrement invocations.

2.2 Writing IRSpecs with QRef

We implemented a prototype tool QRef that takes a concurrent object implementation (as an AST), and verifies bijection, orthogonality, and coverage. A user can work with QRef to interactively create the IRSpecs necessary for a concurrent object. At each step, a user provides a next IRSpec, and QRef checks whether bijection holds, and whether this IRSpec is orthogonal w.r.t. those previously provided. Also, if not enough IRSpecs have been given, QRef will detect this via a failed coverage check reporting a counterexample that illustrates a path through a method not yet accounted for.

Checking bijection, orthogonality, and coverage requires that we first generate various automata and then perform various operations on them. Our implementation in OCaml consumes the object implementation and candidate IRSpecs and then constructs a number of automata including (i) IRSpecs path descriptions (R, N, L) (ii) the suffix-closure and prefix-closure of each IRSpec for orthogonality, (iii) automata of the forms $\Sigma^* \cdot A \cdot \Sigma^*$ and $\Sigma^*_{\neg A} \cdot A \cdot \Sigma^*_{\neg A}$ for bijectivity, and (iv) a starred-union of IRSpec paths for coverage. Next, QRef constructs the necessary automaton operations such intersection, emptiness, containment for each of the conditions. Finally, we use `libvata` [36] to perform the operations, and CVC5 [2] to produce counterexamples to coverage.

We used QRef to create IRSpecs for several canonical and difficult concurrent objects including the Michael-Scott queue [41], Treiber's Stack [49], the Hendler *et al.* Elimination Stack [25], and the Harris *et al.* Restricted Double-Compare, Single-Swap (RDCSS) [23]. As an example, the 9 IRSpecs for the Michael-Scott queue are shown in Fig. 4 (and the rest are in Apx. C of the supplement). For example, the IRSpec $(\mathtt{f}, \mathtt{atom}(\mathtt{abcde} \triangleright \mathtt{f} \triangleleft \mathtt{g}))$ covers the behavior of a succesful dequeue operation, with abcdefg being the "happy path." IRSpec $(\{\mathtt{f}\}, \mathtt{coop}(((\mathtt{abcde})+(\mathtt{abcd})) \triangleright \mathtt{abcdefg} \triangleleft (\mathtt{h}+\mathtt{j})\mathtt{s}))$ then covers the paths that are consequently impacted. Another 7 IRSpecs are needed to cover dequeue, dequeue on an empty queue, and the "helping" feature of MSQ where other threads discover a stale tail pointer and advance it. Overall, our experimental results show that, for each canonical object, all of the object's automaton conditions could be checked in less than 30 seconds.

3 Preliminaries

Kleene Algebra with Tests (KAT). KAT [31] offers an algebraic framework for modeling and reasoning about programs, which we adopt in this work to model concurrent object operations.

Definition 3.1. A KAT is a pair $(\mathcal{K}, \mathcal{B})$, where $\mathcal{K} = (K, +, \cdot, ^*, 0, 1)$ is a Kleene Algebra and $\mathcal{B} = (B, +, \cdot, ^-, 0, 1)$ is a boolean algebra of tests, together with a homomorphic embedding of $\mathcal{B}$ into $\mathcal{K}$. The set of KAT terms is generated from a finite alphabet of primitive actions A and primitive tests B, its syntax being given by the following grammar $KAT \ni \mathsf{k} ::= a \in A \mid b \in B \mid 0 \mid 1 \mid \mathsf{k} + \mathsf{k} \mid \mathsf{k} \cdot \mathsf{k} \mid \mathsf{k}^*$.

KAT expressions model the behavior of concurrent objects by representing statements as actions and conditionals as tests. Multiplication corresponds to sequential composition, addition to nondeterministic choice, and the Kleene star to iteration. We further conservatively extend KAT with the notation $[b \cdot a]$ to syntactically encode atomic operations consisting of a primitive test b followed immediately by a primitive action a, with no other actions in between. While it introduces a restriction of interleavings (defined below), by considering an identity relation for the two symbols [and], the semantics is not impacted.

We use $\preceq_{seq}$ to denote the sequential sub-expression relation on KAT terms, that is, $k_1 \preceq_{seq} k_2$ whenever k_2 accepts a factorization $k' k_1 k''$ for some k', k''.

Concurrent objects. A concurrent object O is defined by a finite set operations that can be invoked concurrently by multiple threads. Each operation $\mathrm{op}(\overrightarrow{x})\backslash\overrightarrow{r}.k_{op} \in O$ is given by its signature, consisting of a unique name op, a list of formal parameters $\overrightarrow{x}$, and a distinguished list of return variables $\overrightarrow{r}$ used to record the result, and an implementation k_{op} encoded as a KAT expression.

Let $\mathcal{T}$ denote the set of thread identifiers. We write Var and Val for the countably infinite set of variables and the set of values, respectively, and we define a state over a finite set of variables $V \subseteq Var$ as a function $\sigma_V : V \to Val$. The set of all such states is denoted by S_V. Let XG be the set of global variables and let XL_t be the set of local variables associated with thread $t \in \mathcal{T}$. We write $XL_\cup = \bigcup_{t\in\mathcal{T}} XL_t$ for the set of all local variables. We assume w.l.o.g. that $XG \cap XL_\cup = \emptyset$ and $XL_{t_i} \cap XL_{j_j} = \emptyset$ for all distinct threads $t_i \neq t_j$. For convenience, we write $\sigma_{X,Y}$ for a state projected onto variables $X \cup Y$, σ_G for σ_{XG}. All projections we consider include global variables, and we write σ_t for σ_{XG,XL_t}.

We model the behavior of concurrent objects in terms of a labeled transition system $\mathcal{M}_O = (\mathcal{C}, \mathcal{C}_{init}, \to, Ev)$, under a most general client that makes an arbitrary number of invocation from an arbitrary number of threads. A configuration $(\sigma_{XG,XL}, T) \in \mathcal{C}$ consists of a state over global and local variables, $\sigma_{XG,XL}$, and a thread pool $T : \mathcal{T} \to KAT \cup \{\bot\}$ capturing the remaining computation per thread, where $\bot$ represents the end of code. When clear from the context, we omit the subscript of the state component and simply write σ for $\sigma_{XG,XL}$. We write $st(C)$ and $kont(C)$ for the state σ and the thread pool T components of C, respectively. The transition relation $\to \subseteq \mathcal{C} \times \mathbb{E} \times \mathcal{C}$ is nondeterministic and it involves a set of events $\mathbb{E} \subseteq \mathcal{T} \times \mathcal{L}$. For simplicity, we write $C \xrightarrow{e} C'$ to denote $(C, e, C') \in \to$. An event e is a thread-label pair (t, l), with labels being drawn from the set $\mathcal{L} = A \cup B \cup \{inv(op, \overrightarrow{v}), res(op, \overrightarrow{r})\} \cup \{[b \cdot a] \mid b \in B, a \in A\}$ of actions, tests, invocation of operations, their responses, and atomic read-modify-write operations (e.g., compare-and-swap). We use $\twoheadrightarrow$ to denote the transitive closure of $\to$. We write $C \overset{e_1 \cdots e_n}{\twoheadrightarrow} C'$ to mean that there exist configurations $C_0, C_1, \ldots, C_n$ such that $C = C_0$, $C' = C_n$, and $C_{i-1} \xrightarrow{e_i} C_i$ for every $1 \leq i < n$. An execution of L is a sequence of alternating configurations and events, $\rho = C_0 e_1 C_1 e_2 \ldots$, starting from an initial configuration $C_0 \in \mathcal{C}_{init}$, such that $C_i \xrightarrow{e_{i+1}} C_{i+1}$ for all i. A configuration C is reachable, denoted by $Reach(C)$, if there exist an execution ending in C. An execution is complete if it ends in a configuration C such that $kont(C)(t) = \bot$ for all $t \in \mathcal{T}$. Projecting out the configurations of an execution results in a *trace*, and the projection of a trace onto the label domain produces a *path*. We denote the universe of traces by $Traces = \mathbb{E}^*$, and write $Traces(O) \subseteq Traces$ for the trace language of an object O. We usually write $\tau \in \mathbb{E}^*$ for a trace and $\pi \in \mathcal{L}^*$ for a path.

Words, languages. We denote by $L_t : \Sigma^* \to (\mathcal{T} \times \Sigma)^*$ the homomorphism that tags every symbol of a word in the language of L with the thread ID t. The (concatenation) product of languages is defined in the usual way as $LL' = \{uu' \mid u \in L \text{ and } u' \in L'\}$. We write $\overrightarrow{\Pi}_{t\in J} L_t$ (resp. $\overleftarrow{\Pi}_{t\in J} L_t$) for the ordered (resp. reverse-ordered) product of thread-indexed languages L_t. We use the notation L^k for the k-fold product of L. For an alphabet Σ, we denote by $\text{Any} = \Sigma^*$ the set of all finite words over Σ. We write $\text{pref}(L)$ and $\text{suf}(L)$ to denote the prefix and suffix closures, respectively, of a language L.

Contexts. A context is a state predicate $B_V : S_V \to \mathbb{B}$ whose free variables are a finite set $V \subseteq Var$. We write $B_{X,Y}$ to denote a context over the union of variables $X \cup Y$. The projection of a context B_V onto $W \subseteq V$ is defined by existential abstraction $\text{proj}_W(B_V) = \exists U.B_V$, where $V = W \uplus U$. We assume that context variables always include the global variables XG, and for brevity we omit them. Accordingly, we write B_t instead of B_{XG,XL_t}, and $\text{proj}_t(B_V)$ for the projection of B_V onto $XG \cup XL_t$. We write $\sigma_t \vDash B_t$ when the state σ_t satisfies the context B_t.

Trace equivalence. Let $S \subseteq \mathcal{L} \times \mathcal{L}$ be a set of swapping labels, for example pairs of actions that operate on thread-local state or access the global state without performing conflicting updates. This relation determines an irreflexive and symmetric relation $I \subseteq (\mathcal{T} \times \mathcal{L}) \times (\mathcal{T} \times \mathcal{L})$, called an **independence relation**, defined as $I = \{(a_{t_i}, b_{t_j}) \mid (a, b) \in S \wedge t_i \neq t_j\}$. Intuitively, two thread-labeled actions are independent when their labels belong to the swapping relation S and they are

executed by different threads. Following *Mazurkiewicz trace theory* [10], an independence relation I induces an **equivalence on traces** generated by commuting adjacent independent thread-labeled actions. We write $\sim_I$ for the smallest equivalence relation on traces closed under such commutations. Two traces related by $\sim_I$ differ only by a series of swaps of independent events and represent equivalent scheduling behaviors.

We denote by $\approx_G$ a **strong equivalence on traces**. Two traces are $\approx_G$-equivalent when the configurations they traverse during an execution have the same projections onto the global variables. It is immediate that $\approx_G$ subsumes $\sim_I$.

Contextual semi-commutativity. A thread may observe an identical local context relating its local state to a given global state S_G at multiple points across an execution, whenever intervening environment transitions don't change the global state S_G or it is changed but later restored. We capture this phenomenon as a relation of coarse contextual semi-commutativity. This relation builds on the semantics of Lipton's movers [39] by restricting the movement of actions according to the observed local contexts. Later, we lift this relation to KAT expressions.

Let $\mathcal{U} \subseteq \{\triangleright, \triangleleft\} \times \Sigma \times P$ be a collection of hypotheses, called **contextual left/right-moving facts**, in the style of Lipton's movers. We write $a \triangleright B$ for $(\triangleright, a, B) \in \mathcal{U}$ and $B \triangleleft a$ for $(\triangleleft, a, B)$ to denote that action a is a right mover and a left mover, respectively, with respect to the context B. Here, B is an assertion over thread-local state and the global state of the object, universally quantified over thread identifiers. We consider relations $\mathcal{U}$ that satisfy a sound coarse semi-commutativity. That is, for all $a \triangleright B$:

$$\forall C_1, C_2, C_3 \in \mathcal{C}. \forall \sigma \in S_{XG,XL}. \forall t \in \mathcal{T}. \forall \tau \in (\mathbb{E} \cap ((\mathcal{T} \setminus \{t\}) \times \mathcal{L}))^*.\ Reach(C_1) \wedge \\ C_1 \xrightarrow{a_t} C_2 \overset{\tau}{\twoheadrightarrow} C_3 \ \wedge\ st(C_3) = \sigma \ \wedge\ \sigma_t \vDash B_t \implies (\exists\, C_4 \in \mathcal{C}.\ C_1 \overset{\tau}{\twoheadrightarrow} C_4 \xrightarrow{a_t} C_3) \ \wedge\ a_t\tau \approx_G \tau a_t$$

and for all $B \triangleleft a$:

$$\forall C_1, C_2, C_3 \in \mathcal{C}. \forall \sigma \in S_{XG,XL}. \forall t \in \mathcal{T}. \forall \tau \in (\mathbb{E} \cap ((\mathcal{T} \setminus \{t\}) \times \mathcal{L}))^*.\ Reach(C_1) \wedge \\ C_1 \overset{\tau}{\twoheadrightarrow} C_2 \xrightarrow{a_t} C_3 \ \wedge\ st(C_1) = \sigma \ \wedge\ \sigma_t \vDash B_t \implies (\exists\, C_4 \in \mathcal{C}.\ C_1 \xrightarrow{a_t} C_4 \overset{\tau}{\twoheadrightarrow} C_3) \ \wedge\ \tau a_t \approx_G a_t\tau$$

Intuitively, the above conditions require that the thread actions semi-commute with environment transitions whenever the context B_t holds. The first clause states that if, from a reachable configuration C_1 a thread t runs the action a_t followed by a sequence of transitions performed by other threads leading to configuration C_3 whose state satisfies the thread t-local context B_t, then the same effect can be obtained by first executing the environment transitions and then running the action a_t. Additionally, we require that $a_t\tau$ and τa_t are strongly equivalent, ensuring that any action enabled in τ remaining enabled after reordering. The second condition captures the dual case of the left-moving action a_t under an reachable configuration C_1 whose state satisfies the thread-local context B_t.

These facts induce a **rewrite relation** on traces describing how thread-labeled actions may be reordered toward canonical patterns.

Let $\subsetneq_{\mathcal{U}}$ be the smallest preorder on traces induced by $\mathcal{U}$ satisfying the following rules:

(i) $\tau'\, a_t \tau B_t\, \tau'' \subsetneq_{\mathcal{U}} \tau'\, \tau a_t B_t\, \tau''$ whenever $a \triangleright B$,

(ii) $\tau'\, B_t \tau a_t\, \tau'' \subsetneq_{\mathcal{U}} \tau'\, B_t a_t \tau\, \tau''$ whenever $B \triangleleft a$.

That is, contextual semi-commutativity moves actions across execution segments produced by other threads whenever the contextual conditions guarantee that the global behavior of the object is preserved. Unlike the independence relation, contextual semi-commutativity is directional.

Reductions. To address the problem of exponentially many interleavings in concurrent systems, prior work [14, 20, 22] has shown that **reduction** techniques provide an effective framework by restricting attention to representative traces while preserving observable behavior. Intuitively,

reductions exploit commutativity properties of actions to reorder executions into canonical forms. We build on this idea and formalize a normalization relation $\prec$ on traces that captures rewritings modulo swap of independent events. Formally, we say that $\tau_1 \prec \tau_2$ whenever there exist traces τ', τ'', such that:

$$\tau_1 \sim_I \tau' \ \wedge \ \tau' \subsetneq_{\mathcal{U}} \tau'' \ \wedge \ \tau'' \sim_I \tau_2$$

The relation $\prec$ captures the effect of applying contextual mover rewrites while commuting independent events. Its reflexive-transitive closure $\leq = (\prec)^*$ represents sequences of normalization steps. Since both independence and contextual semi-commutativity steps preserve the global behavior of executions, we have $\tau_1 \approx_G \tau_2$ whenever $\tau_1 \leq \tau_2$.

For a language of traces T, its *downward closure* is defined as:

$$\downarrow T = \{\tau' \mid \exists \tau \in T . \tau' \leq \tau\}$$

In other words, the downward closure of T contains all traces whose canonical form is in T.

Definition 3.2 (Reduction). Let O and O' be two concurrent objects with their respective trace languages $Traces(O)$ and $Traces(O')$. We say that O' is a *reduction* of O, denoted by $O' \sqsubseteq O$, if $Traces(O) \subseteq \downarrow Traces(O')$

4 Contextual Semi-Commutativity over KAT

We shift focus to the source code level and introduce an inductive definition of *contextual commutativity over KAT expressions*. While trace-based (semi-)commutativity allows us to capture when actions within concrete executions may be safely reordered, reasoning at the level of traces is inconvenient for verification, since it abstracts away the syntactic structure of code and the way actions are composed. Therefore, we lift the notion of contextual semi-commutativity to the language used to describe concurrent object implementations. We define it inductively over the syntax of KAT expressions, propagating semi-commutativity through sequential composition, choice, and iteration in a manner that respects the trace semantics.

Since a relation $\mathcal{U}$ defines semi-commutativity for single actions and KAT expressions represent languages of control paths, we need to lift the relation first to sequences of thread-labeled actions and then to languages of such sequences. To this end, we begin by defining Dijsktra predicate transformers over traces, allowing contexts to be propagated along traces and provide reasoning about languages of traces via nondeterministic choice. There are two types depending on the direction of propagation: weakest precondition for backward moving and strongest postcondition for forward moving. We assume semantic functions for actions and tests denoted by $[\![a]\!] : S_{XG,XL} \to S_{XG,XL}$ and $[\![b]\!] : S_{XG,XL} \to \mathbb{B}$, respectively, where the action/test are defined over global variables XG and local variables XL. We lift these interpretations to thread-indexed actions and tests by defining, for each thread t, $[\![a_t]\!] : S_{XG,XL_t} \to S_{XG,XL_t}$ and $[\![b_t]\!] : S_{XG,XL_t} \to \mathbb{B}$. Let $\tau = e_1 e_2 \ldots e_n$ be a finite sequence of events executed by various threads, with its semantic function given by the sequential composition $[\![\tau]\!] = [\![e_1]\!] \circ \cdots \circ [\![e_n]\!]$. We define the weakest precondition transformer $\mathrm{wp}(\tau, B)(\sigma) = \ \forall \ \sigma' \in \Sigma_V . \sigma' = [\![\tau]\!](\sigma) \Rightarrow B(\sigma')$, and the strongest postcondition transformer $\mathrm{sp}(\tau, B)(\sigma') = \ \exists \ \sigma \in \Sigma_V . B(\sigma) \wedge [\![\tau]\!](\sigma) = \sigma'$. Then, given a language L of finite traces, we take $\mathrm{wp}(L, B)$ (resp. sp) by computing the predicate transformers of the nondeterministic choice over the traces in L. Finally, for any KAT expression k representing a language of paths, we define: $\mathrm{wp}_t(\mathsf{k}, B) = \bigwedge_{\pi \in [\![\mathsf{k}]\!]} \mathrm{wp}(\pi_t, B_t)$ and $\mathrm{sp}_t(\mathsf{k}, B) = \bigvee_{\pi \in [\![\mathsf{k}]\!]} \mathrm{sp}(\pi_t, B_t)$.

Judgment $\mathsf{k}_t \triangleright B_t$ means that KAT expression k executed by thread t is *right-moving with respect to context* B_t: any path represented by k can be permuted one action at a time to the right of other threads' events up to the context B_t.

$$\textsc{Action}\ \frac{(\triangleright, a, B) \in \mathcal{U}}{a_t \triangleright B_t} \qquad \textsc{Test}\ \frac{b_t \subseteq B_t}{b_t \triangleright B_t} \qquad \textsc{Choice}\ \frac{(\mathsf{k}_1)_t \triangleright B_t \qquad (\mathsf{k}_2)_t \triangleright B_t}{(\mathsf{k}_1 + \mathsf{k}_2)_t \triangleright B_t} \qquad \textsc{Star}\ \frac{\mathsf{k}_t \triangleright B_t \qquad B_t \subseteq \mathrm{wp}_t(\mathsf{k}, B)}{(\mathsf{k}^*)_t \triangleright B_t}$$

$$\textsc{Seq}\ \frac{(\mathsf{k}_2)_t \triangleright B_t \qquad B'_t \subseteq \mathrm{wp}_t(\mathsf{k}_2, B) \qquad (\mathsf{k}_1)_t \triangleright B'_t}{(\mathsf{k}_1\mathsf{k}_2)_t \triangleright B_t}$$

Fig. 1. Contextual Commutativity over KAT expressions

A primitive action a executed by thread t is right-moving with respect to B_t whenever there is a corresponding hypothesis in $\mathcal{U}$. Then, since tests b do not modify state, they commute whenever they are consistent with the context. The rule for the nondeterministic choice $(\mathsf{k}_1 + \mathsf{k}_2)_t$ requires that each branch is right-moving with respect to the same context. For a sequence $(\mathsf{k}_1\mathsf{k}_2)_t$ to be right-moving up to a context B_t, we reason backwardly. The rule relates each expression to its own context, while the weakest-precondition constraint ensures that the states required for right-commuting $(\mathsf{k}_1)_t$ preserve the enabledness of $(\mathsf{k}_2)_t$. Lastly, an iteration $(\mathsf{k}^*)_t$ is right-moving with respect to a context B_t when each k_t is right-moving to B_t and all initial states from which k can be executed by thread t are within the same footprint as B_t.

5 Induction-Reduction Specifications: IRSpecs

In this section we introduce *induction–reduction specifications* (IRSpecs), the central construct of our methodology. IRSpecs describe atomic segments and interaction patterns between threads, providing a structured framework for selecting canonical executions. An induction-reduction specification, IRSpec, connects inductively quantified actions recurring across traces of a concurrent objects with a coarse-grained Lipton reduction that contracts right/left-moving control paths to a non-moving anchor. We distinguish two main classes of IRSpecs that we discuss next.

The first is the *atomic* (atom) class which represents blocks of instructions executed by a single thread without interference. An atomic IRSpec $\alpha = (\mathfrak{a}, \mathrm{atom}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L})) \in \hat{A} \times KAT^3$ consists of an inductively quantified primitive action or primitive atomic operation $\mathfrak{a} \in \hat{A} = A \cup \{[ba] \mid b \in B, a \in A\}$ and three KAT expressions R, N, L coarsely representing right-moving paths, anchor paths, and left-moving paths, respectively. Similarly to a routine definition in [39], α specifies indivisible sequences of actions in the language of RNL that are executed in isolation, each corresponding to an occurrence of the recurring action $\mathfrak{a}$ in a trace.

The second, called the *cooperative* (coop) class, captures the optimistic synchronization protocol inherent in lock-free concurrent objects, where many threads attempt an update of the shared state; one of them succeeds, while the others restart the operation. Its focus is on the paths executed by the later failing threads. A cooperative IRSpec $\alpha = (\mathfrak{a}^{\cup}, \mathrm{coop}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L})) \in 2^{\hat{A}} \times KAT^3$ centers around the reason(s) for failure, represented by N, of their attempts. For this class of IRSpecs, the anchor N typically denotes a nondeterministic choice among subexpressions representing atomic segments, each corresponding to global-state update. A path represented by N is executed by a given thread and causes another thread, after executing a path represented by R, to continue with the fallback path in L, possibly leading to a restart of the retry loop. The inductive component of the cooperative specification considers now a set of actions $\mathfrak{a}^{\cup}$, each associated to a reason for failing the update.

IRSpecs order. Because atomic IRSpecs describe paths inside an object methods that are scheduled to run uninterruptedly, they are used in specifying code segments centering a global state mutation.

These global state updates serve as failure reasons inside the cooperative IRSpecs. Consequently, there is an inherent dependence relation of IRSpecs that we capture via a precedence order.

Given two IRSpecs, α_1 and α_2 we say that α_1 *precedes* α_2 and we write $\alpha_1 \prec \alpha_2$ if the following conditions are satisfied:

(i) $\alpha_1 = (\mathfrak{a}_1, \mathrm{atom}(\mathrm{R}_1 \triangleright \mathrm{N}_1 \triangleleft \mathrm{L}_1))$ and $\alpha_2 = (\mathfrak{a}_2^{\cup}, \mathrm{coop}(\mathrm{R}_2 \triangleright \mathrm{N}_2 \triangleleft \mathrm{L}_2))$
(ii) $\mathfrak{a}_1 \in \mathfrak{a}_2^{\cup}$ and $\mathrm{R}_1\mathrm{N}_1\mathrm{L}_1 \leq \mathrm{N}_2$

Note that this definition specifies the precedence relation only between atomic and cooperative IRSpecs. Lastly, an *admissible linear extension* $\prec_1$ of $\prec$ is a total order on $\mathcal{A}$ such that the identity map on $\mathcal{A}$ is order-preserving from $(\mathcal{A}, \prec)$ to $(\mathcal{A}, \prec_1)$.

Example 5.1. Recall the counter example from Section 2, where we informally related the *atomic* and *cooperative* IRSpecs. For the cooperative increment IRSpec α_{incF}, we require $\alpha_{incS} \prec \alpha_{incF}$ and $\alpha_{decS} \prec \alpha_{incF}$. These constraints state that the atomic blocks of α_{incS} and α_{decS} are admissible candidates for anchors in the induction-reduction steps of α_{incF}. In this way, the order relates a cooperative IRSpec to the atomic IRSpecs that may justify its reductions.

Language of canonical trace expressions. We next introduce a language of *canonical trace expressions* for describing the canonical forms of interleavings induced by IRSpecs. Let *CanonExp* denote the set of expressions generated by the grammar: $r ::= [\mathsf{k}] \mid \langle\langle \mathsf{k}[\mathsf{k}]\mathsf{k} \rangle\rangle \mid r + r \mid r \cdot r \mid r^*$, where k ranges over KAT expressions. Intuitively, these expressions describe structured fragments of executions that arise after organizing thread interleavings according to the reduction component of IRSpecs.

The two basic constructs correspond to the two classes of IRSpecs. The expression $[\mathsf{k}]$ represents *atomic fragments* in which a single thread executes a control path described by k without interference from other threads. The expression $\langle\langle \mathsf{k}_1[\mathsf{k}_2]\mathsf{k}_3 \rangle\rangle$ captures *cooperative interaction patterns*. Specifically, an unbounded number of threads execute, each in isolation, paths described by k_1, are interrupted by another thread executing a path in k_2, and then continue with their respective isolated executions of paths described by k_2. Composite expressions are formed using the standard regular expression operators of choice, sequencing, and Kleene star. It is straightforward to represent the reduction components of IRSpecs using *CanonExp*. An atomic specification $\alpha = (\mathfrak{a}, \mathrm{atom}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L}))$ has the corresponding canonical expression $[\mathrm{RNL}]$, while a cooperation specification $\alpha = (\mathfrak{a}^{\cup}, \mathrm{coop}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L}))$ has $\langle\langle \mathrm{R}[\mathrm{N}]\mathrm{L} \rangle\rangle$ as its corresponding expression. For an IRSpec α, we write $canon(\alpha)$ to refer to its canonical trace expression.

Semantics of canonical trace expressions. A canonical trace expression r characterizes a restriction on interleavings that conform to a particular interaction pattern between threads. We define its semantics $(\!| r |\!)$ as a context-free language of trace fragments, building on the path semantics $[\![\mathsf{k}]\!]$ of KAT expressions:

$$(\!|\, [\mathsf{k}] \,|\!) \triangleq \bigcup_{t \in \mathcal{T}} [\![\mathsf{k}]\!]_t \qquad (\!|\, \langle\langle \mathsf{k}_1[\mathsf{k}_2]\mathsf{k}_3 \rangle\rangle \,|\!) \triangleq \bigcup_{t \in \mathcal{T}} \bigcup_{J \subseteq \mathcal{T} \setminus \{t\}} (\overrightarrow{\Pi}_{t_j \in J} [\![\mathsf{k}_1]\!]_{t_j}) [\![\mathsf{k}_2]\!]_t (\overleftarrow{\Pi}_{t_j \in J} [\![\mathsf{k}_3]\!]_{t_j})$$

$$(\!| r_1 + r_2 |\!) \triangleq (\!| r_1 |\!) \cup (\!| r_2 |\!) \qquad (\!| r_1 \cdot r_2 |\!) \triangleq (\!| r_1 |\!)(\!| r_2 |\!) \qquad (\!| r^* |\!) \triangleq \bigcup_{n \in \mathbb{N}} (\!| r |\!)^n$$

The semantics of atomic expressions $(\!|\, [\mathsf{k}] \,|\!)$ is the language of trace fragments in which a single thread executes a path described by k.

The semantics of cooperative expressions $(\!|\, \langle\langle \mathsf{k}_1[\mathsf{k}_2]\mathsf{k}_3 \rangle\rangle \,|\!)$ captures interactions where a distinguished thread executes a path k_2 while an arbitrary set of other threads execute surrounding paths k_1 and k_3. Composite expressions follow the standard language interpretation of regular expression operators. For any canonical trace expression $r \in CanonExp$, we write $\mathrm{Any}_{\neg r} = \mathrm{Any} \setminus (\!| r |\!)$ to denote the complement of traces described by r.

Lemma 5.2 (Distribution over anchor choice). *Let* $\mathrm{N}_\cup$ *be a finite set of KAT expressions. Then*

$$\langle\!\langle R[\sum_{\mathrm{N}\in\mathrm{N}_\cup} N]L\rangle\!\rangle \;=\; \sum_{N\in N_\cup} \langle\!\langle R[N]L\rangle\!\rangle$$

Proof. By unfolding the semantics of cooperative canonical expressions and using that concatenation distributes over union. □

Canonical traces. For a concurrent object O and a canonical trace expression r we write $O[r]$ for the abstraction of O obtained by restricting its executions to those consistent with r. Thus, $O[r]$ does not introduce new behaviors. It retains only those executions of O whose interleavings can be organized according to the block structure prescribed by r In this sense, $O[r]$ provides a coarser execution semantics for O abstracting away from fine-grained interleavings while preserving the interaction discipline encoded by r. The traces arising from executions of $O[r]$ are called **canonical traces** of O with respect to r.

Definition 5.3 (Preference order). Let r be a canonical trace expression, let $L = \mathrm{Any}_{\neg r}(|r|)$. For a trace τ, its *r-rank* defined as $\mathrm{rank}_r(\tau) = \max\{k \mid \tau \in L^k\mathrm{Any}\}$, represents the number of canonical r-fragments occurring in the prefix of τ. We define the preference order on traces induced by r as follows:

$$\tau_1 \preceq_r \tau_2 \;\text{ iff }\; \mathrm{rank}_r(\tau_1) \leq \mathrm{rank}_r(\tau_2)$$

We write $\tau_1 \prec_r \tau_2$ for the corresponding strict order. Lexicographic orderings on traces have been used [10, 17] to define reduction techniques in concurrent program verification, notably for symmetry breaking. Adopting a similar idea, we define reductions whose traces are r-canonical, i.e., traces having maximal r-rank. We later establish the conditions under which r-maximality within each equivalence class corresponds to the strongest sound restriction on traces.

Semantics of IRSpecs. To define the semantics of IRSpecs, we rely on the equivalence relation $\approx_G$ introduced earlier that identifies executions modulo their global-state updates. Traces that differ only in the reordering of thread-local actions but induce the same evolution of global-state belong to the same equivalence class. We write $[\tau]$ for the equivalence class of τ in a trace set T, i.e., $[\tau] = \{\tau' \in T \mid \tau' \sim \tau\}$. Given an induction-reduction specification α and its corresponding canonical expression $r = canon(\alpha)$, the semantics $[\![\alpha]\!] : \mathcal{P}(Traces) \to \mathcal{P}(Traces)$ can naturally be defined as a canonicalization operator.

For a given set of traces T, $[\![\alpha]\!](T)$ selects from each equivalence class the traces that are maximal with respect to preference order $\preceq_r$ induced by canonical expression r. We formally define it as:

$$[\![\alpha]\!](T) = \{max_{\preceq_r}[\tau] \mid \tau \in T\}$$

Algebraically, $[\![\alpha]\!]$ behaves as an interior operator on the poset $(\mathcal{P}(T), \subseteq)$, i.e., it is monotone, idempotent and reductive.

Remark 1. *Let O be a concurrent object, and let O' be the object whose trace set is $[\![\alpha]\!](Traces(O))$ for some IRSpec α. Then $O' \sqsubseteq O$.*

We write $O' \overset{\alpha}{\sqsubseteq} O$ for the reduction of O via α.

While both $O[r]$ and $[\![\alpha]\!](Traces(O))$ are determined by the same canonical expression $r = canon(\alpha)$, they capture different notions. The abstraction $O[r]$ is syntactic. It restricts the executions to those that satisfy the block discipline prescribed by r. In contrast $[\![\alpha]\!](Traces(O))$ is semantic, retaining representative traces within their equivalent classes. Accordingly, $O[r]$ describes traces of canonical forms, whereas $[\![\alpha]\!](Traces(O))$ describe traces that survive IRSpec-induced reduction, and equality between the two requires additional conditions and does not hold in general. For

example, when the expression R (and/or L) of the IRSpec represents control paths that cannot be soundly moved immediately before (and/or after) the anchor. Because actions along these paths may be interleaved with actions executed by other threads that are not swappable, the IRSpec fails to restrict T to r-canonical traces. To address these limitations, we later introduce sufficient conditions on IRSpec that guarantee a proper restriction.

A degenerate case arises when the right- and left-mover components of an IRSpec are both ϵ. In this case, the induced canonicalization imposes no additional normalization, and hence $[\![\alpha]\!]$ acts as the identity on trace sets. Moreover, if the anchor itself is a primitive synchronization operation such as CAS, introducing no new block structure beyond the original execution semantics of the object, then the corresponding abstraction $O[r]$ also coincides with O. Thus, such IRSpecs do not yield progress and serve only as trivial specifications.

Composition of IRSpecs. Since IRSpecs act on sets of traces their sequential composition has a natural interpretation as function composition. Specifically, we define the semantics of $\alpha_1;\alpha_2$ as $[\![\alpha_1;\alpha_2]\!] = [\![\alpha_2]\!] \circ [\![\alpha_1]\!]$. The composition is partial, as it is undefined when the order of the specifications does not satisfy their precedence order.

6 Proof Rules with Syntactic Premises

In this section we present proof rules for establishing that the traces selected by an IRSpec are precisely the canonical traces prescribed by the canonical expression of its reduction component. For an IRSpec α with $r = canon(\alpha)$, the rules soundly ensure that $[\![\alpha]\!](Traces(O)) = Traces(O[r])$. In this way, they characterize when the semantic reduction induced by α coincides with the canonical block structure described by r. To make this correspondence checkable, the rules reduce the induction-reduction argument to automation-friendly primitives: Hoare logic, contextual commutativity, and syntactic automaton operations on control-path structure. We begin by introducing a few auxiliary notions used in their formulation.

The proof rules reason only about the control paths that actively participate in a reduction. For an IRSpec α, we denote by $[\alpha]$ its **reduction pattern**, defined as the KAT expression formed from its reduction components. Specifically, $[(\mathfrak{a}, \mathrm{atom}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L}))] = \mathrm{RNL}$ and $[(\mathfrak{a}^{\cup}, \mathrm{coop}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L}))] = \mathrm{RL}$.

When several IRSpecs are used together, their reduction patterns must not interfere. This is captured by **orthogonality**. Given two induction-reduction specification α_i and α_j, we say their reduction patterns are *orthogonal*, written $[\alpha_1]$ *orthog* $[\alpha_2]$, when the following conditions hold:

(i) *non-overlapping*: $\mathrm{suf}([\alpha_i]) \cap \mathrm{pref}([\alpha_j]) = \emptyset$ and $\mathrm{pref}([\alpha_i]) \cap \mathrm{suf}([\alpha_j]) = \emptyset$

(ii) *non-factoring*: $[\alpha_i] \not\leq_{seq} [\alpha_j]$ and $[\alpha_j] \not\leq_{seq} [\alpha_i]$

Intuitively, orthogonality ensures that distinct induction-reductions do not compete for the same control path. The first condition requires that any two control-paths subject to two IRSpecs respectively, do not partially overlap, while the second condition excludes cases in which one control-path is subpath of another. A reader familiar with string rewriting systems may notice the analogy with the sufficient condition for confluence.

A second non-interference condition of reduction patterns is **non-self-overlap**. Formally, for a specification α, we write *non-self-ovl*$([\alpha])$ to mean that $\mathrm{suf}([\alpha]) \cap \mathrm{pref}([\alpha]) = \emptyset$.

Definition 6.1. Let r_1 and r_2 be canonical expressions for a concurrent object O. We overload the reduction relation and write $r_2 \sqsubseteq r_1$ to mean $O[r_2]$ is a reduction of $O[r_1]$. Furthermore, if $r_2 = canon(\alpha)$ for some IRSpec α, we write $r_2 \overset{\alpha}{\sqsubseteq} r_1$ to indicate that the reduction from $O[r_1]$ to $O[r_2]$ is induced by α, i.e., $O[r_2] \overset{\alpha}{\sqsubseteq} O[r_1]$.

In general, the canonical abstraction $(O[r_1])[r_2]$ and the semantic operator $[\![\alpha]\!](Traces(O[r_1]))$ need not coincide. To this end, the rules below capture the sufficient conditions under which they

$$\textsc{Atom}\ \frac{\mathfrak{a} \overset{t_i}{\longleftrightarrow} \mathrm{RNL} \qquad \textit{non-self-ovl}(RNL) \qquad \{B_{t_i}\}\mathrm{N}_{t_i}\{B'_{t_i}\} \qquad \mathrm{R} \triangleright_{t_i} B \qquad B'_{\ t_i} \triangleleft \mathrm{L}}{[\mathrm{N}] \overset{\alpha}{\sqsupseteq} [\mathrm{RNL}]}\ [\forall\, t_i \in \mathcal{T}\,]$$

$$\textsc{ASeq}\ \frac{[N_1] \overset{\alpha_1}{\sqsupseteq} [R_1N_1L_1] \qquad [N_2] \overset{\alpha_2}{\sqsupseteq} [R_2N_2L_2] \qquad R_1N_1L_1\ \textit{orthog}\ R_2N_2L_2}{[N_1 + N_2] \overset{\alpha_1;\alpha_2}{\sqsupseteq} [R_1N_1L_1 + R_2N_2L_2]}$$

Fig. 2. Proof rules for reductions induced by *Atom* IRSpecs

coincide. Since our presentation starts from a concrete concurrent implementation and derives progressively more sequential abstractions, the canonical expression allowing a higher degree of interleaving appears on the left-hand side of $\sqsupseteq$.

The rule Atom is universally quantified by the tread ID, t_i, and is used to establish an *atomicity reduction* induced by $(\mathfrak{a}, \mathrm{atom}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L}))$. Intuitively, it captures the fact that all executions in which the control paths represented by RNL are performed by a thread t_i and interleaved with statements from concurrent threads can be simulated by executions in which these paths can be executed in isolation. The premise $\mathfrak{a} \overset{t_i}{\longleftrightarrow} \mathrm{RNL}$ is a shorthand notation for a trace-indexed family of bijective and order preserving functions $\forall\, \tau \in T.\ \mathrm{occ}_{\mathfrak{a}}(\tau|_{t_i}) \leftrightarrow \mathrm{occ}_{\mathrm{RNL}}(\tau|_{t_i})$ which map occurrences of induction action to occurrences of control paths described by RNL in the thread restriction of the trace. Since atom reductions express coarse-grained atomicity within each method's implementation, this condition can be argued about through syntactic lenses at the level of the object's methods. For instance, the conditions: *(i)* $\mathrm{RNL} \leq \mathrm{Any}_{\neg\mathfrak{a}}\ \mathfrak{a}\ \mathrm{Any}_{\neg\mathfrak{a}}$ and *(ii)* $\forall\, \alpha' \in \mathcal{A}.\ \alpha' \neq \alpha \Rightarrow [\alpha'] \cap (\mathrm{Any}\ \mathfrak{a}\ \mathrm{Any}) = 0$ are sufficient for establishing $\mathfrak{a} \leftrightarrow \mathrm{RNL}$. We defer the full details of the proof to the Appendix A. The premise *non-self-ovl*(RNL) excludes self-overlap of the reduction pattern RNL. Specifically, it rules out nonempty proper sequences of actions that are both suffixes and prefixes of RNL. This guarantees that occurrences of the atomic block do not overlap inside a trace. The validity of the premise $\{B_{t_i}\}N_{t_i}\{B'_{t_i}\}$, called a *stable Hoare triple*, entails two properties. First, for any configuration whose state satisfies B_{t_i}, every state reachable by executing a path $\pi_N \in [\![N]\!]$ by thread t_i satisfies B'_{t_i}. Second, for every reachable configuration in which N is enabled for thread t_i, its corresponding state satisfies B_{t_i}. Formally, we say that $\{B_{t_i}\}N_{t_i}\{B'_{t_i}\}$ holds if and only if $\forall\sigma.\ B_{t_i}(\sigma) \Rightarrow \bigvee_{\pi_N \in [\![N]\!]}(\forall\sigma'.\ \sigma' = [\![(\pi_N)_{t_i}]\!](\sigma) \Rightarrow B'_{t_i}(\sigma'))$ and *(ii)* $\forall\sigma.\ \textit{enabled}(N, \sigma) \Rightarrow B_{t_i}(\sigma)$, where $\textit{enabled}(N, \sigma)$ holds if and only if there exists a path $\pi_N \in [\![N]\!]$ that is executable by t_i from σ. The last two premises, $\mathrm{R} \triangleright_{t_i} B$ and $B'_{\ t_i} \triangleleft \mathrm{L}$ establish the contextual commutativity conditions for the induction-reduction candidate's right- and left-mover components.

Consider for the counter, the IRSpec α_{incS} which corresponds to a successful increment of the counter. The contexts B_{t_i} and B'_{t_i} capture the local view of thread t_i. Concretely, B_{t_i} is $c_{t_i} = ctr$, representing states where thread t_i observes a local copy of the global counter, and B'_{t_i} is $\exists\, oldCtr.\ ctr = oldCtr + 1 \wedge c_{t_i} = oldCtr$, indicating that the global counter has been incremented while the local value c_{t_i} observed by thread t_i still reflects the pre-update state. See Apx D.1 for the contexts used in the rules for counter.

$$\textsc{Coop}\ \frac{\begin{array}{c}\mathfrak{a}^{\cup} \overset{t_i}{\longleftrightarrow} \mathrm{N} \qquad t_j \neq t_i \qquad C_{t_j} = \mathrm{sp}_{t_j}(\mathrm{R}, \mathit{true}) \qquad C'_{t_j} = \mathrm{wp}_{t_j}(\mathrm{L}, \mathit{true}) \\ \{C_{t_j}\}\mathrm{N}_{t_i}\{C'_{t_j}\} \qquad \mathit{IStable}(C_{t_j}, \overline{bk(N)}) \qquad bk(N) \overset{t_i}{\leftrightarrow} N \\ \textit{non-self-ovl}(RL) \qquad \mathrm{R} \triangleright_{t_j} C \qquad C'_{t_j} \triangleleft \mathrm{L} \qquad \mathrm{R} \circlearrowleft \mathrm{R} \qquad \mathrm{L} \circlearrowleft \mathrm{L}\end{array}}{[\mathrm{N}] \overset{\alpha}{\sqsupseteq} \langle\!\langle \mathrm{R}[\mathrm{N}]\mathrm{L} \rangle\!\rangle}\ [\forall\, t_i, t_j \in \mathcal{T}]$$

$$\textsc{CStruct}\ \frac{[\sum_{N \in N^{\cup}}] \overset{\alpha}{\sqsupseteq} \langle\!\langle R[\sum_{N \in N^{\cup}} N]L \rangle\!\rangle}{[N] \overset{\alpha}{\sqsupseteq} \langle\!\langle R[N]L \rangle\!\rangle} \qquad \textsc{CSeq}\ \frac{\begin{array}{c}[N] \overset{\alpha_1}{\sqsupseteq} \langle\!\langle R_1[N]L_1 \rangle\!\rangle \qquad [N] \overset{\alpha_2}{\sqsupseteq} \langle\!\langle R_2[N]L_2 \rangle\!\rangle \\ R_1L_1 \ \textit{orthog}\ R_2L_2 \qquad R_1 \circlearrowleft R_2 \qquad L_1 \circlearrowleft L_2\end{array}}{[N] \overset{\alpha_1;\alpha_2}{\sqsupseteq} \langle\!\langle (R_1 + R_2)[N](L_1 + L_2) \rangle\!\rangle}$$

Fig. 3. Proof rules for reduction induced by *Coop* IRSpecs

Lemma 6.2 (Soundness of atomic reductions). *Let $\mathcal{A}$ be a collection of IRSpecs for a concurrent object O. The rules Atom and ASeq are sound for all* atomic *IRSpecs in $\mathcal{A}$.*

The proofs are relegated to Appendix A.

The rule ASeq composes two atomic reductions into a single one. Each premise states that executions centered around an anchor N_i can be reduced to the corresponding atomic block $\mathrm{R}_i\mathrm{N}_i\mathrm{L}_i$. The orthogonality side condition ensures that the two atomic patterns do not overlap or interfere, so that the corresponding reductions can be applied independently. Under this assumption, the conclusion combines them by nondeterministic choice. Specifically, an execution whose anchor is drawn from $\mathrm{N}_1 + \mathrm{N}_2$ is reduced to the corresponding canonical block drawn from $R_1N_1L_1 + R_2N_2L_2$. Thus, ASeq expresses closure of orthogonal atomic reductions.

The rule Coop establishes a *cooperative reduction* induced by an IRSpec of the form $(\mathfrak{a}^{\cup}, \mathrm{coop}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L}))$. Its conclusion states that executions centered around the anchor N can be reordered into a cooperative block in which some thread t_i executes N, while another arbitrary thread $t_j \neq t_i$ executes a path R before the anchor and continues with a path in L afterwards. The premise $\mathfrak{a}^{\cup} \overset{t_i}{\leftrightarrow} \mathrm{N}$ requires a bijective correspondence between occurrences of inductive actions in the set $\mathfrak{a}^{\cup}$ and occurrences of N in the trace restriction to thread t_i. For every arbitrary thread $t_j \neq t_i$, the contexts $C_{t_j} = \mathrm{sp}_{t_j}(\mathrm{R}, \mathit{true})$ and $C'_{t_j} = \mathrm{wp}_{t_j}(\mathrm{L}, \mathit{true})$ summarize the effect of the surrounding paths. Namely, C_{t_j} characterizes the states reachable after executing a path in R, while C'_{t_j} characterizes the states from which a path in L can be executed. Then, the Hoare-style premise $\{C_{t_j}\}N_{t_i}\{C'_{t_j}\}$ states that execution of the anchor N by thread t_i transforms the context established by R into one from which the fallback path L is enabled for thread t_j.

For instance, in the case of the counter "coop" IRSpec α_{incF}, the contexts C_{t_j} and C'_{t_j} capture the state local to thread t_j. Specifically, C_{t_j} is $c_{t_j} = ctr$, representing states in which the thread t_j observes, via its local variable c_{t_j}, the current value of the global counter. The post-context C'_{t_j} is $c_{t_j} \neq ctr$, indicating that the global counter has changed since that observation. The reason is given by the anchor N executed by a different thread t_i. This anchor N represents a nondeterministic choice between the paths `int c = ctr; cas(ctr, c, c+1); return c` and `int c = ctr; cas(ctr, c, c-1); return c`, which, when executed by other threads, lead to the change in the global counter observed by t_j. See Apx. D.1 for all contexts used in the rules for counter.

The premise $IStable(C_{t_j}, \overline{bk(N)})$ requires that C_{t_j} be stable under interference from all actions except those in N that correspond to a bijective support of N executed by threads other than t_j. Namely, the helper $bk(N)$ extracts the actions in N whose occurrences in a trace projection to a thread t_i are in bijective correspondence with occurrences of N-paths in that projection. This correspondence is captured by the condition $bk(N) \overset{t_i}{\leftrightarrow} N$. The complement $\overline{bk(N)}$ thus denotes the set of all primitive actions with those corresponding to $bk(N)$ removed. Formally, given a context C_{t_j} and a set of actions Act, we define the interference stability condition as:

$$IStable(C_{t_j}, Act) \triangleq \forall a \in Act,\ t_k \in \mathcal{T}.\ t_k \neq t_j \Rightarrow \{\, C_{t_j} \wedge pre(a_{t_k}) \,\}\ a_{t_k}\ \{\, C_{t_j} \,\},$$

where $pre(a_{t_k})$ denotes the state assertion characterizing the precondition under which action a is enabled for thread t_k. This condition ensures that once the context C_{t_j} has been established between two occurrences of paths in N, it cannot be invalidated by unrelated interference. Consequently, any transition from C_{t_j} to a distinct context C'_{t_j} can only be caused by an action in $bk(N)$, and hence arises exclusively from occurrences of paths in N. While not explicitly stated, $\mathfrak{a}^{\cup}$ is typically a subset of $bk(N)$, as the inductive actions of a cooperative IRSpec usually coincide with a distinguished subset of the anchor's bijective support.

The premise *non-self-ovl*(RL) excludes self-overlapping reduction patterns RL corresponding to the cooperative specification. The contextual commutativity conditions $R \triangleright_{t_j} C$ and $C'\ {}_{t_j}\!\triangleleft L$ cover paths executed by threads $t_j \neq t_i$ and establish their conditions under which they can be moved into position around the anchor. Finally, the conditions $R \rightleftarrows R$ and $L \rightleftarrows L$ require that actions occurring in paths of R, and respectively, L, commute pairwise, so that their interleavings can be freely reordered.

The rule CStruct specializes a cooperative reduction whose anchor is given as a nondeterministic choice. It states that if a cooperative reduction holds for an anchor of the form $\sum_{N \in N^{\cup}} N$, then the same reduction may be understood component-wise for each $N \in N^{\cup}$. In this way, CStruct reduces reasoning about a composite anchor to reasoning about its individual cases.

The rule CSeq composes two cooperative reductions around the same anchor N. Each of the two reduction premises captures one class of surrounding behaviors, namely paths in R_i before the anchor, and paths in L_i afterwards. The orthogonality side condition ensures that their corresponding paths do not overlap, while the commutativity conditions $R_1 \rightleftarrows R_2$ (resp. $L_1 \rightleftarrows L_2$) require their respective actions to be swappable. The conclusion then merges them into a single cooperative block $\langle\!\langle (R_1 + R_2)[N](L_1 + L_2) \rangle\!\rangle$, expressing closure of cooperative reductions under nondeterministic combination of compatible surrounding paths.

Lemma 6.3 (Soundness of cooperative reductions). *Let $\mathcal{A}$ be a collection of IRSpecs for a concurrent object O. The rules Coop, CStruct, and CSeq are sound for all* cooperative *IRSpecs in $\mathcal{A}$.*

The proofs are relegated to Appendix A.

Covering of induction-reductions. Let $\mathcal{A}$ be a collection of IRSpecs for a concurrent object O. We say that $\mathcal{A}$ is **covering** for O if

(i) for each operation implementation k_{op}, there exists $\mathcal{A}_{\mathrm{op}} = \{\alpha_1, \ldots, \alpha_l\} \subseteq \mathcal{A}$ that *covers* it, i.e., $\mathsf{k}_{\mathrm{op}} \leq ([\alpha_1] + \cdots + [\alpha_l])^*$

(ii) for every cooperative IRSpec $\alpha = (\mathfrak{a}^{\cup}, \mathrm{coop}(R \triangleright N \triangleleft L))$ in $\mathcal{A}_{\mathrm{op}}$, there exists a set of atomic IRSpecs $C_a \subseteq \mathcal{A}_{\mathrm{op}}$ such that $\forall\ \alpha_a \in C_a.\ \alpha_a \prec \alpha$ and

$$\mathfrak{a}^{\cup} = \{\mathfrak{a} \mid \exists\, (\mathfrak{a}, \mathrm{atom}(\ldots)) \in C_a\} \ \wedge\ N = \sum \{[\alpha_a] \mid \alpha_a \in C_a\}$$

The first condition requires that any repetitive partial control-flow path of the implementation be described by the reduction pattern of some IRSpec in $\mathcal{A}_{\mathrm{op}}$. The second condition requires that

every cooperative specification is anchored in a collection of preceding atomic specifications, so that its failure scenarios are aligned with concrete successful executions.

6.1 Canonicalization of an object

Let $\mathcal{A}$ be a collection of IRSpecs for a concurrent object O, and let

$$\prec_1 \triangleq \alpha_1; \alpha_2; \cdots; \alpha_n$$

be an admissible linear extension of their precedence order. The *canonicalization* of O induced by $\prec_1$ is the object $O_{can}^{\prec_1}$ obtained by applying the corresponding reductions in sequence:

$$O \overset{\alpha_1}{\sqsupseteq} O_1 \overset{\alpha_2}{\sqsupseteq} \ldots \overset{\alpha_n}{\sqsupseteq} O_{can}^{\prec_1}$$

Intuitively, $O_{can}^{\prec_1}$ is the abstraction of O obtained by retaining only the traces that survive canonicalization under the IRSpecs ordered by $\prec_1$.

Theorem 6.4 (Closed form of the canonical traces). *Let O be a concurrent object, and let $\mathcal{A}$ be a collection of IRSpecs that is covering and orthogonal for O. If $\prec_1$ is an admissible linear extension of the precedence order on $\mathcal{A}$, then $O_{can}^{\prec_1} \sqsubseteq O$ and $(r_1 + \cdots + r_m)^*$ is an abstraction of the $O_{can}^{\prec_1}$, that is:*

$$Traces(O_{can}^{\prec_1}) \subseteq (r_1 + \cdots + r_m)^*,$$

where each r_i is a basic reduction derived from the rules.

The proof details are available in Appendix A, but we summarize here. The covering condition gives a decomposition of every operation of path of O into IRSpec patterns, while orthogonality guarantees that this decomposition is unambiguous and non-interfering. The soundness of the reduction rules then let us replace each such pattern by its corresponding canonical trace expression, one reduction at a time. Since canonical trace expressions are closed under choice and concatenation, the whole trace belongs to the starred union of the basic canonical expressions.

7 Implementation

The proof rules in Sec. 6 reduce IRSpec reasoning to conditions that are automation-friendly. We implemented a prototype tool in OCaml called QRef[1] that automatically proves the bijectivity (conditions (i) and (ii)), orthogonality (non-overlapping and non-factoring), and coverage. QRef consumes an input object with method implementations as symbol-annotated ASTs (annotation can be done automatically), and some candidate IRSpecs. The method implementations and IRSpec KAT expressions (R, N, L) are all translated to automata. Finally, QRef constructs the automata queries discussed in Sec. 6 and uses `libvata` [36] to perform the automaton operations. When a coverage test fails, QRef generates a counter-example in the form of a word in the method that is not accounted for by an IRSpec (using CVC5 [2]).

Interactivity. The automated checking of conditions can guide a user, helping them write IRSpecs, as we illustrate using MSQ in Sec. 8. As a user writes each additional IRSpec, QRef will report errors that instruct the user what to do nexts. If QRef reports a bijectivity violation in one of the spec tuples, the user must repair that tuple. If QRef reports an orthogonality violation between two specs α, α', the user must narrow one of them (or add them to the partial order). Finally, if QRef reports a lack of coverage for a given method, the user has not written enough IRSpecs and must create another one for the path of the method that was not yet covered. To help the user, QRef generates a counterexample path, which provides a hint as to method behaviors not yet considered.

[1]We plan to publicly release QRef. The QRef source code and case study implementations are available in the supplemental materials.

Object	# of IRSpecs	Aut. Gen Time	Orthog. Q	Orthog. Time	Bijection Q	Bijection Time	Coverage Q	Coverage Time	Total Time
Counter	4	0.0	24	0.09	18	0.06	2	0.01	4.21
Elim. Stack	12	11.4	264	1.13	166	0.71	2	0.04	25.26
Treiber's Stack	5	0.2	40	0.17	30	0.11	2	0.01	5.44
Michael-Scott Queue	9	3.0	144	0.52	96	0.41	3	0.04	12.94
RDCSS	10	0.7	180	0.66	95	0.43	1	0.01	11.81

Table 1. Results of applying QRef to verify IRSpecs for various canonical object implementations. All times are in seconds.

8 Case Studies

We evaluated (i) whether IRSpecs could be written and (ii) whether the orthogonality, bijectivity and coverage conditions could be automatically validated. To that end, we used QRef to guide the process of writing and validating IRSpecs for several canonical examples from the literature:

- Treiber's Stack [49]. Supports unboundedly many threads performing concurrent push and pop operations.
- Hendler *et al.* Elimination Stack [25]. Uses Treiber as a basic object and provides another layer of synchronization where a thread attempting a pop can consume the value arriving from a concurrent thread attempting a push. This involves multiple CAS operations on two arrays (one to announce thread descriptors and another to perform the "collision").
- Michael-Scott Queue [41]. A non-blocking queue that supports unboundedly many threads concurrently enqueuing/dequeuing.
- Harris *et al.* Restricted Double-Compare, Single-Swap (RDCSS) [41]. A non-blocking object consisting of a Data register and a Control register, with threads attempting to "write" by updating the Data register, but only if both Data & Control have an expected value. Threads may "help" if one thread has begun but not completed its write operation.

Our experiments were performed inside an Ubuntu 24.04 Parallels guest OS running on a MacBook M1 Pro with 32 GB of RAM and 10 cores. The results are summarized in Table 1. For each object, we report the number of **IRSpecs** required, the time for **Aut**omata **Gen**eration (in seconds), as well as the number of queries and execution time for each of the checks: **Orthogonality** (non-overlapping/non-factoring), **Bijection**, and **Coverage**.

All IRSpecs for these objects are given in Apx. C. These IRSpecs were written with the help of QRef (see example below) and in all of the cases, a dozen or fewer were necessary, confirming research question (i). Furthermore, QRef was able to validate the final versions of these IRSpecs of all of the objects within a matter of seconds, confirming research question (ii).

Example: MSQ. Recall the implementation of MSQ shown in the top of Fig. 4. We identify basic statements and conditions with black circles (e.g. ⓐ) which will occur in the IRSpecs. (Some local variable definitions omitted for lack of space.) The MSQ is a linked list from global pointers `Q.head` and `Q.tail`, and manipulated as follows. Values are stored in the nodes between `Q.head` and `Q.tail`, with `enq` adding new elements to the `Q.tail`, and `deq` removing elements from `Q.head`. During a successful CAS in `enq`, the `Q.tail→next` pointer is changed from null to the new node. However, this new item cannot be dequeued until `adv` advances `Q.tail` forward to point to the new node. A `deq` on an empty list (when `Q.head=Q.tail`) returns immediately. Otherwise, `deq` attempts to advance `Q.head` and, if success, returns the value in the now-omitted node. The original MSQ implementation includes the `adv` CAS inside `enq` and `deq` iterations. We have done this for expository purposes and it is not necessary.

```
int enq(int v){ loop {
 ⓐnode_t *node=...;
 ⓑtail=Q.tail; ⓒnext=tail→next;
 if (ⓓQ.tail==tail) {
  if (ⓔnext==null) {
   if (ⓕCAS(&tail→next,next,node))
      ⓖreturn 1;
   else ⓗ[cas/f] ⓢskip;
  } else ⓙ[next≠null] ⓢskip;
 } else ⓚ[Q.tail≠tail] ⓢskip; } }

void adv() { loop {
 ⓠtail=Q.tail; next=tail→next;
 if (ⓡnext≠null) {
  if (ⓢCAS(&Q→tail,tail,next))
   ⓣreturn 0;
  else ⓤ[cas/f] ⓥskip;
 } else ⓦ[next==null] ⓧskip; } }
```

```
int deq(){ loop {
 Ⓐint pval;
 head=Q.head;tail=Q.tail;
 next=head→next;
 if (ⒷQ.head==head) {
  if (Ⓒhead==tail) {
   if (Ⓓnext==null) Ⓔret 0;
   else Ⓕ[next≠null] Ⓖskip;
  } else Ⓗ[head≠tail] {
    Ⓘpval=next→val;
    if (ⒿCAS(&Q→head,head,next))
      Ⓚret pval;
    else Ⓛ[cas/f] Ⓜskip;
  } } }
} else Ⓝ[Q.head≠head] Ⓞskip;
}
```

$$
\begin{aligned}
\alpha_{\text{fAEnqS}} &= (a_i\!:\!\mathtt{f}, \quad \mathtt{atom\{abcde \triangleright f \triangleleft g\}})\\
\alpha_{\text{fCEnqF}} &= (a_i\!:\!\mathtt{f}, \quad \mathtt{coop\{((abcde)+(abcd)) \triangleright abcdefg \triangleleft (h+j)s\}})\\
\alpha_{\text{fADeqS}} &= (a_i\!:\!\mathtt{J}, \quad \mathtt{atom\{ABCHI \triangleright J \triangleleft K\}})\\
\alpha_{\text{fCDeqF}} &= (a_i\!:\!\mathtt{J}, \quad \mathtt{coop\{((ABCHI)+(AN)) \triangleright ABCHIJK \triangleleft ((LM)+(O))\}})\\
\alpha_{\text{fAEmp}} &= (a_i\!:\!\mathtt{D}, \quad \mathtt{atom\{ABC \triangleright D \triangleleft E\}})\\
\alpha_{\text{fAAdvS}} &= (a_i\!:\!\mathtt{S}, \quad \mathtt{atom\{QR \triangleright S \triangleleft T\}})\\
\alpha_{\text{fCAdvF}} &= (a_i\!:\!\mathtt{S}, \quad \mathtt{coop\{((QR)+(abc)) \triangleright QRST \triangleleft ((UV)+(ks))\}})\\
\alpha_{\text{fAAdv}} &= (a_i\!:\!\mathtt{W}, \quad \mathtt{atom\{Q \triangleright W \triangleleft X\}})\\
\alpha_{\text{fAOne}} &= (a_i\!:\!\mathtt{F}, \quad \mathtt{atom\{ABC \triangleright F \triangleleft G\}})
\end{aligned}
$$

with order: $\alpha_{\text{fAEnqS}} \prec \alpha_{\text{fCEnqF}}, \alpha_{\text{fADeqS}} \prec \alpha_{\text{fCDeqF}}, \alpha_{\text{fAAdvS}} \prec \alpha_{\text{fCAdvF}}$

Fig. 4. The Michael-Scott Queue and its nine IRSpecs.

Interactively writing IRSpecs for MSQ. All unbounded thread behaviors of the MSQ are covered by the nine IRSpecs shown at the bottom of Fig. 4. As we describe these specs, we also explain how QRef can guide a user to create them and use the proof rules (Sec. 6) to validate them. The first spec α_{fAEnqS} is the "happy path" of the enqueuer, which performs reads and validations as it passes from ⓐ-ⓔ, then performing a successful CAS ⓕ and returning ⓖ. This specifies that induction should range over all successful CAS ⓕ occurrences and, for each, reduce the trace using the L and R to make ⓐ-ⓖ an atomic section. QRef will verify that bijection and orthogonality hold (largely easy because there are not yet any other IRSpecs), but coverage will fail and report the counterexample "abcdehs." This next path occurs when other threads are racing against the happy path, but fail their CAS operations. Thus a user can write IRSpec α_{fCEnqF}, order it with α_{fAEnqS}, and specify those other threads' paths that fail. In this case there are two such paths (the second one will subsequently be given by QRef): one that reaches CAS failure ⓗ and another which goes from ⓓ to ⓙ.

The dequeue operations has analogous IRSpecs α_{fADeqS} for the "happy path" ABCHIJK of the dequeuer and α_{fCDeqF} for the two paths that are impacted: another dequeuer attempting a CAS, or merely discovering at Ⓝ that its read of Q.head is now stale. If the user omits the $\alpha_{\text{fADeqS}} \prec \alpha_{\text{fADeqF}}$ order constraint, QRef reports a bijection condition (ii) violation. QRef will still find coverage not to hold, with path ABCDE unaddressed, so a user adds α_{AEmp} to cover the case where a dequeuer finds that the queue is empty and returns.

The remaining IRSpecs cover scenarios pertaining to the helper thread that advances the tail pointer with adv. α_{fAAdvS} and α_{fCAdvF} provide the cases where an advancer thread successfully performs CAS at Ⓢ, which impacts other advancers (racing at Ⓢ) as well as other *enqueuer* threads that find the tail pointer to be stale via path abcks. The last case α_{fAOne} pertains to a helper thread finding that there is nothing to be done because the tail is not lagged.

QRef reports that these 9 IRSpecs cover the entirety of MSQ. The left/right moverness of these actions are up to their respective anchors which we have manually written in Apx. D. (We leave integration with contextual commutativity synthesis tools [7, 8] for future work.) As an example, for α_{fAEnqS} the context B includes information such as $tail_{t_i} \rightarrow next = next_{t_i}$ and post-CAS context B' includes that $Q.tail \rightarrow next = node_{t_i}$. The enqueue happy path prefix abcde can move to the right up to B, and suffix g can move to the left to B'. Meanwhile, α_{fCEnqF} has additional context C for the failing cases, with information such as $tail_{t_j} \rightarrow next = next_{t_j}$ but in context C' that $Q.tail \rightarrow next \neq node_{t_j}$. Those coop path prefixes/suffixes can similarly move to the right/left.

9 Related work

The core contribution of this paper is to show that scenario-based reasoning can be automated through induction-reduction specifications and proof rules that reduce the problem to Hoare logic, contextual commutativity and automaton operations (for orthogonality, bijectivity and coverage). To our knowledge no prior works provide such automation. We here build upon the "scenario-based" reasoning of Enea and Koskinen [16]. Although their work set out key definitions and described manual scenario-based proofs, it did not provide a means of mechanizing such proofs, and the implementation merely produces candidate (unsound) so-called quotient expressions.

Program *logics* for concurrent programs have been studied extensively, as mentioned in Sec. 1. Improving on classical Owicki-Gries [45] and Rely-Guarantee [27] logics, numerous extensions of Concurrent Separation Logic [3, 5, 43, 46] have been proposed to reason compositionally about different instances of fine-grained concurrency, e.g. [9, 13, 28, 29, 35, 37, 42, 47, 48, 50–52].

Other earlier approaches to concurrent reasoning have also been based on the *reduction theory* of Lipton [38], who introduced the concept of left/right *movers* that re-order operations to create atomic blocks of code. Tools such as QED [15] and CIVL [24] expanded upon Lipton's theory and added invariant reasoning and refinement layers [33, 34]. Reasoning via simplifying program transformations has also been adopted in the context of mechanized proofs, e.g., [6]. Inductive sequentialization [32] builds upon this prior work, and introduces a new scheme for reasoning inductively over unbounded concurrent executions.

The works of Farzan et al. [17, 18] define an algorithmic verification framework for concurrent programs that includes computing sound reductions. The syntactic representation of these reductions is somewhat complex, which is fine in the context of a fully automated, non-interactive proof. In our paper, we look at classes of reductions which are humanly readable and use simple syntactic constructs.

10 Conclusion

We have shown a way of expressing a combination of the powerful reasoning techniques of induction and reduction, and those so-called IRSpecs can be validated through known reasoning primitives: Hoare logic, contextual commutativity, and syntactic automaton conditions. A handful of IRSpecs allow unbounded thread behavior on a concurrent object to be expressed as a few tangible atom/coop "scenarios." Furthermore the proof rules, which are embodied in our implementation QRef, allow a user to interactively write IRSpecs, receiving feedback when they are incorrect or insufficient.

Future work. We plan to build a more mature version of QRef, and integrate with existing methods for Hoare logic [1] and commutativity synthesis [8]. Although commutativity tools mostly focus on single-action commutativity, algorithms can be developed to extend to our more coarse-grained commutativity up to a context. Futher in the future we will explore whether IRSpecs can be synthesized.

11 Data-Availability Statement

The supplemental materials includes the extended version of the paper with appendices. Also included is the source code of the software supporting the results, scripts for reproducing the experiments, and a README with instructions for building and running the tool.

References

[1] 2020. Boogie. https://github.com/boogie-org/boogie

[2] Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 13243)*, Dana Fisman and Grigore Rosu (Eds.). Springer, 415–442. doi:10.1007/978-3-030-99524-9_24

[3] Richard Bornat, Cristiano Calcagno, Peter W. O'Hearn, and Matthew J. Parkinson. 2005. Permission accounting in separation logic. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2005, Long Beach, California, USA, January 12-14, 2005*. 259–270. doi:10.1145/1040305.1040327

[4] Stephen Brookes. 2007. A semantics for concurrent separation logic. *Theor. Comput. Sci.* 375, 1-3 (2007), 227–270. doi:10.1016/j.tcs.2006.12.034

[5] Stephen D. Brookes. 2004. A Semantics for Concurrent Separation Logic. In *CONCUR 2004 - Concurrency Theory, 15th International Conference, London, UK, August 31 - September 3, 2004, Proceedings*. 16–34. doi:10.1007/978-3-540-28644-8_2

[6] Tej Chajed, M. Frans Kaashoek, Butler W. Lampson, and Nickolai Zeldovich. 2018. Verifying concurrent software using movers in CSPEC. In *OSDI*. https://www.usenix.org/conference/osdi18/presentation/chajed

[7] Adam Chen, Parisa Fathololumi, Eric Koskinen, and Jared Pincus. 2022. Veracity: declarative multicore programming with commutativity. *Proc. ACM Program. Lang.* 6, OOPSLA2 (2022), 1726–1756. doi:10.1145/3563349

[8] Adam Chen, Parisa Fathololumi, Mihai Nicola, Jared Pincus, Tegan Brennan, and Eric Koskinen. 2023. Better Predicates and Heuristics for Improved Commutativity Synthesis. In *Automated Technology for Verification and Analysis - 21st International Symposium, ATVA 2023, Singapore, October 24-27, 2023, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 14216)*, Étienne André and Jun Sun (Eds.). Springer, 93–113. doi:10.1007/978-3-031-45332-8_5

[9] Pedro da Rocha Pinto, Thomas Dinsdale-Young, and Philippa Gardner. 2014. TaDA: A Logic for Time and Data Abstraction. In *ECOOP 2014 - Object-Oriented Programming - 28th European Conference, Uppsala, Sweden, July 28 - August 1, 2014. Proceedings*. 207–231. doi:10.1007/978-3-662-44202-9_9

[10] Volker Diekert and Grzegorz Rozenberg (Eds.). 1995. *The Book of Traces*. World Scientific. doi:10.1142/2563

[11] Thomas Dinsdale-Young, Lars Birkedal, Philippa Gardner, Matthew J. Parkinson, and Hongseok Yang. 2013. Views: compositional reasoning for concurrent programs. In *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*, Roberto Giacobazzi and Radhia Cousot (Eds.). ACM, 287–300. doi:10.1145/2429069.2429104

[12] Mike Dodds, Xinyu Feng, Matthew J. Parkinson, and Viktor Vafeiadis. 2009. Deny-Guarantee Reasoning. In *Programming Languages and Systems, 18th European Symposium on Programming, ESOP 2009, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009, York, UK, March 22-29, 2009. Proceedings (Lecture Notes in Computer Science, Vol. 5502)*, Giuseppe Castagna (Ed.). Springer, 363–377. doi:10.1007/978-3-642-00590-9_26

[13] Cezara Dragoi, Ashutosh Gupta, and Thomas A. Henzinger. 2013. Automatic Linearizability Proofs of Concurrent Objects with Cooperating Updates. In *CAV '13 (LNCS, Vol. 8044)*. Springer, 174–190.

[14] Tayfun Elmas, Shaz Qadeer, Ali Sezgin, Omer Subasi, and Serdar Taşıran. 2010. Simplifying Linearizability Proofs with Reduction and Abstraction. In *TACAS '10: Proc. 16th Intl. Conf. on Tools and Algorithms for the Construction and Analysis of Systems (LNCS, Vol. 6015)*. Springer, 296–311.

[15] Tayfun Elmas, Shaz Qadeer, and Serdar Tasiran. 2009. A calculus of atomic actions. In *POPL*. doi:10.1145/1480881.1480885

[16] Constantin Enea and Eric Koskinen. 2024. Scenario-Based Proofs for Concurrent Objects. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 1294–1323. doi:10.1145/3649857

[17] Azadeh Farzan, Dominik Klumpp, and Andreas Podelski. 2022. Sound sequentialization for concurrent program verification. In *PLDI '22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*, Ranjit Jhala and Isil Dillig (Eds.). ACM, 506–521. doi:10.1145/3519939.3523727

[18] Azadeh Farzan, Dominik Klumpp, and Andreas Podelski. 2024. Commutativity Simplifies Proofs of Parameterized Programs. *Proc. ACM Program. Lang.* 8, POPL (2024), 2485–2513. doi:10.1145/3632925

[19] Azadeh Farzan and Umang Mathur. 2024. Coarser Equivalences for Causal Concurrency. *Proc. ACM Program. Lang.* 8, POPL (2024), 911–941. doi:10.1145/3632873

[20] Azadeh Farzan and Anthony Vandikas. 2020. Reductions for safety proofs. *Proc. ACM Program. Lang.* 4, POPL (2020), 13:1–13:28. doi:10.1145/3371081

[21] Cormac Flanagan and Patrice Godefroid. 2005. Dynamic partial-order reduction for model checking software. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2005, Long Beach, California, USA, January 12-14, 2005*, Jens Palsberg and Martín Abadi (Eds.). ACM, 110–121. doi:10.1145/1040305.1040315

[22] Patrice Godefroid. 1996. *Partial-Order Methods for the Verification of Concurrent Systems - An Approach to the State-Explosion Problem*. Lecture Notes in Computer Science, Vol. 1032. Springer. doi:10.1007/3-540-60761-7

[23] Timothy L. Harris, Keir Fraser, and Ian A. Pratt. 2002. A Practical Multi-word Compare-and-Swap Operation. In *Distributed Computing, 16th International Conference, DISC 2002, Toulouse, France, October 28-30, 2002 Proceedings (Lecture Notes in Computer Science, Vol. 2508)*, Dahlia Malkhi (Ed.). Springer, 265–279. doi:10.1007/3-540-36108-1_18

[24] Chris Hawblitzel, Erez Petrank, Shaz Qadeer, and Serdar Tasiran. 2015. Automated and Modular Refinement Reasoning for Concurrent Programs. In *CAV*. doi:10.1007/978-3-319-21668-3_26

[25] Danny Hendler, Nir Shavit, and Lena Yerushalmi. 2004. A scalable lock-free stack algorithm. In *SPAA 2004: Proceedings of the Sixteenth Annual ACM Symposium on Parallelism in Algorithms and Architectures, June 27-30, 2004, Barcelona, Spain*, Phillip B. Gibbons and Micah Adler (Eds.). ACM, 206–215. doi:10.1145/1007912.1007944

[26] Maurice Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (1990), 463–492. doi:10.1145/78969.78972

[27] Cliff B. Jones. 1983. Specification and Design of (Parallel) Programs. In *IFIP Congress*. 321–332.

[28] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. doi:10.1017/S0956796818000151

[29] Ralf Jung, Rodolphe Lepigre, Gaurav Parthasarathy, Marianna Rapoport, Amin Timany, Derek Dreyer, and Bart Jacobs. 2020. The future is ours: prophecy variables in separation logic. *Proc. ACM Program. Lang.* 4, POPL (2020), 45:1–45:32. doi:10.1145/3371113

[30] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*. 637–650. doi:10.1145/2676726.2676980

[31] Dexter Kozen. 1997. Kleene Algebra with Tests. *ACM Trans. Program. Lang. Syst.* 19, 3 (1997), 427–443. doi:10.1145/256167.256195

[32] Bernhard Kragl, Constantin Enea, Thomas A. Henzinger, Suha Orhun Mutluergil, and Shaz Qadeer. 2020. Inductive sequentialization of asynchronous programs. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, Alastair F. Donaldson and Emina Torlak (Eds.). ACM, 227–242. doi:10.1145/3385412.3385980

[33] Bernhard Kragl and Shaz Qadeer. 2018. Layered Concurrent Programs. In *CAV*. doi:10.1007/978-3-319-96145-3_5

[34] Bernhard Kragl, Shaz Qadeer, and Thomas A. Henzinger. 2018. Synchronizing the Asynchronous. In *CONCUR*. doi:10.4230/LIPIcs.CONCUR.2018.21

[35] Siddharth Krishna, Dennis E. Shasha, and Thomas Wies. 2018. Go with the flow: compositional abstractions for concurrent data structures. *PACMPL* 2, POPL (2018), 37:1–37:31. doi:10.1145/3158125

[36] Ondrej Lengal, Jiri Simacek, Tomas Vojnar, Martin Hruska, and Lukas Holik. [n. d.]. libVATA - A C++ library for efficient manipulation with non-deterministic finite (tree) automata. https://github.com/ondrik/libvata/.

[37] Ruy Ley-Wild and Aleksandar Nanevski. 2013. Subjective auxiliary state for coarse-grained concurrency. In *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*. 561–574. doi:10.1145/2429069.2429134

[38] Richard J. Lipton. 1975. Reduction: A Method of Proving Properties of Parallel Programs. *Commun. ACM* 18, 12 (1975). doi:10.1145/361227.361234

[39] Richard J. Lipton. 1975. Reduction: A New Method of Proving Properties of Systems of Processes. In *Conference Record of the Second ACM Symposium on Principles of Programming Languages, Palo Alto, California, USA, January 1975*, Robert M. Graham, Michael A. Harrison, and John C. Reynolds (Eds.). ACM Press, 78–86. doi:10.1145/512976.512985

[40] Antoni W. Mazurkiewicz. 1986. Trace Theory. In *Petri Nets: Central Models and Their Properties, Advances in Petri Nets 1986, Part II, Proceedings of an Advanced Course, Bad Honnef, Germany, 8-19 September 1986 (Lecture Notes in Computer Science, Vol. 255)*, Wilfried Brauer, Wolfgang Reisig, and Grzegorz Rozenberg (Eds.). Springer, 279–324. doi:10.1007/3-540-17906-2_30

[41] Maged M. Michael and Michael L. Scott. 1996. Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. In *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing, Philadelphia, Pennsylvania, USA, May 23-26, 1996*, James E. Burns and Yoram Moses (Eds.). ACM, 267–275. doi:10.1145/248052.248106

[42] Aleksandar Nanevski, Anindya Banerjee, Germán Andrés Delbianco, and Ignacio Fábregas. 2019. Specifying concurrent programs in separation logic: morphisms and simulations. *Proc. ACM Program. Lang.* 3, OOPSLA (2019), 161:1–161:30. doi:10.1145/3360587

[43] Peter W. O'Hearn. 2004. Resources, Concurrency and Local Reasoning. In *CONCUR 2004 - Concurrency Theory, 15th International Conference, London, UK, August 31 - September 3, 2004, Proceedings*. 49–67. doi:10.1007/978-3-540-28644-8_4

[44] Peter W. O'Hearn. 2007. Resources, concurrency, and local reasoning. *Theor. Comput. Sci.* 375, 1-3 (2007). doi:10.1016/j.tcs.2006.12.035

[45] Susan S. Owicki and David Gries. 1976. Verifying Properties of Parallel Programs: An Axiomatic Approach. *Commun. ACM* 19, 5 (1976), 279–285. doi:10.1145/360051.360224

[46] Matthew J. Parkinson, Richard Bornat, and Peter W. O'Hearn. 2007. Modular verification of a non-blocking stack. In *Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2007, Nice, France, January 17-19, 2007*. 297–302. doi:10.1145/1190216.1190261

[47] Azalea Raad, Jules Villard, and Philippa Gardner. 2015. CoLoSL: Concurrent Local Subjective Logic. In *Programming Languages and Systems - 24th European Symposium on Programming, ESOP 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings*. 710–735. doi:10.1007/978-3-662-46669-8_29

[48] Ilya Sergey, Aleksandar Nanevski, and Anindya Banerjee. 2015. Specifying and Verifying Concurrent Algorithms with Histories and Subjectivity. In *Programming Languages and Systems - 24th European Symposium on Programming, ESOP 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings*. 333–358. doi:10.1007/978-3-662-46669-8_14

[49] R. K. Treiber. 1986. *Systems Programming: Coping with Parallelism.* Technical Report RJ 5118. IBM Almaden Research Center.

[50] Aaron Turon, Derek Dreyer, and Lars Birkedal. 2013. Unifying refinement and hoare-style reasoning in a logic for higher-order concurrency. In *ACM SIGPLAN International Conference on Functional Programming, ICFP'13, Boston, MA, USA - September 25 - 27, 2013*. 377–390. doi:10.1145/2500365.2500600

[51] V. Vafeiadis. 2008. *Modular fine-grained concurrency verification.* Ph. D. Dissertation. University of Cambridge.

[52] Viktor Vafeiadis. 2009. Shape-Value Abstraction for Verifying Linearizability. In *VMCAI '09: Proc. 10th Intl. Conf. on Verification, Model Checking, and Abstract Interpretation (LNCS, Vol. 5403)*. Springer, 335–348.

[53] Viktor Vafeiadis and Matthew J. Parkinson. 2007. A Marriage of Rely/Guarantee and Separation Logic. In *CONCUR 2007 - Concurrency Theory, 18th International Conference, CONCUR 2007, Lisbon, Portugal, September 3-8, 2007, Proceedings (Lecture Notes in Computer Science, Vol. 4703)*, Luís Caires and Vasco Thudichum Vasconcelos (Eds.). Springer, 256–271. doi:10.1007/978-3-540-74407-8_18

A Proofs

Lemma A.1. *Let $\mathcal{A}$ be a collection of IRSpecs that are covering and orthogonal for a concurrent object O, and let $\alpha = (\mathfrak{a}, \mathrm{atom}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L})) \in \mathcal{A}$. For each trace τ and each thread t, let*

$$f_{\tau,t} : \mathrm{occ}_{\mathfrak{a}}(\tau|_t) \longrightarrow \mathrm{occ}_{\mathrm{RNL}}(\tau|_t)$$

be an order-preserving bijection. Such a family $\{f_{\tau,t}\}_{\tau,t}$ exists provided the following conditions hold:

(i) $\mathrm{RNL} \leq \mathrm{Any}_{\neg\mathfrak{a}}\ \mathfrak{a}\ \mathrm{Any}_{\neg\mathfrak{a}}$
(ii) $\forall\ \alpha' \in \mathcal{A}.\ \alpha' \neq \alpha \Rightarrow [\alpha'] \cap (\mathrm{Any}\ \mathfrak{a}\ \mathrm{Any}) = 0$

Proof. The proof is by construction. Fix an arbitrary trace subset T in $Traces(O)$. Given the two conditions, we define a family of bijections $\forall\ \tau \in T. \forall\ t \in \mathcal{T}.\ \mathrm{occ}_{\mathfrak{a}}(\tau|_t) \leftrightarrow \mathrm{occ}_{\mathrm{RNL}}(\tau|_t)$ between occurrences of $\mathfrak{a}$ in a trace restriction to thread t and occurrences of control paths represented by the regular expression RNL in the same trace restriction.

Given any word w, we define the occurrence functions occ for a symbol $\mathfrak{a}$ and a regular expression k as follows:

$$\begin{array}{rcl} \mathrm{occ}_a(w) & = & \{i \in \mathbb{N} \mid w[i] = a\} \\ \mathrm{occ}_{\mathsf{k}}(w) & = & \{(i,j) \in \mathbb{N}^2 \mid w[i:j] \in [\![\mathsf{k}]\!]\} \end{array}$$

As expected, for a symbol a, $\mathrm{occ}_a(w)$ returns the set of positions at which a occurs in the string w, while for a regular expression, k, $\mathrm{occ}_k(w)$ returns the set of index pairs identifying factors of w in the language of k.

Let τ be an arbitrary trace in T, let t be an arbitrary thread ID in $\mathcal{T}$.

By the definition of covering and orthogonality of $\mathcal{A}$, there exists a factorization of the restriction of τ to the thread t, $\tau|_t = \tau_0\tau_1 \cdots$ such that, for every τ_k, there exist an induction-reduction α_k with its reduction pattern $pat_k = [\alpha_k]$ and $\tau_k \in [\![pat_k]\!]_t$. That is, τ_k is in the language defined by the reduction pattern of α_k tagged by thread t. Since α is an atomic specification, $[\alpha] = \mathrm{RNL}$. Therefore, the set $\mathrm{occ}_{\mathrm{RNL}}(\tau|_t)$ induces a total interval order. The interval order is defined in the usual way: $(i_1, j_1) \leq (i_2, j_2) \Leftrightarrow j_1 \leq i_2$ and it's strict whenever $j_1 < i_2$.

Define the function $f : \mathrm{occ}_{\mathrm{RNL}}(\tau|_t) \longrightarrow \mathrm{occ}_{\mathfrak{a}}(\tau|_t)$ by:

$$f((i,j)) = l, \text{ where } i \leq l < j \text{ and } \tau|_t(l) = \mathfrak{a}$$

We show that f is a bijection.

From (i) it follows any word in RNL has exactly one occurrence of $\mathfrak{a}$, that is, l exists and it's unique. That ensures that the function f is well defined. It's immediate to see that f is injective, since there is no overlap between RNL-factors in τ_t.

To prove surjectivity of f we proceed by contradiction. Assume that there exists $l \in \mathrm{occ}_{\mathfrak{a}}(\tau|_t)$ that lies outside the RNL-factors of $\tau|_t$. Let τ_l be such factor. By the factorization of $\tau|_t$, it must be that τ_l is in the language of the reduction pattern $pat' = [\alpha']$ of some IRSpec α' that is distinct from α. However this contradicts (ii).

Lastly, f is order-preserving by definition. □

Lemma A.2. *Let $\mathcal{A}$ be a collection of IRSpecs that are covering and orthogonal for a concurrent object O. For any $\alpha = (\mathfrak{a}^{\cup}, \mathrm{coop}(\mathrm{R} \triangleright \mathrm{N} \triangleleft \mathrm{L})) \in \mathcal{A}$, there exists a family of trace- and thread-indexed, order-preserving bijections*

$$\{\mathrm{occ}_{\mathfrak{a}^{\cup}}(\tau|_t) \leftrightarrow \mathrm{occ}_{\mathrm{N}}(\tau|_t)\}_{\tau,t}$$

provided that

(i) for each $\mathfrak{a} \in \mathfrak{a}^{\cup}$, there exists an atomic IRSpec $\alpha_a \in \mathcal{A}$ such that $\alpha_a \prec \alpha$
(ii) $N = \sum k$ for $k \in \{R_a N_a L_a \mid \exists\ (\mathfrak{a}_a, \mathrm{atom}(R_a \triangleright N_a \triangleleft L_a)) \prec \alpha\}$

(iii) for each $\alpha_a = (\mathfrak{a}_a, \mathrm{atom}(R_a \triangleright N_a \triangleleft L_a))$ *such that* $\alpha_a \in \mathcal{A}$ *and* $\alpha_a \prec \alpha$*, there exists a family of trace- and thread-indexed order-preserving bijections* $\{f_{\tau,t} : \mathrm{occ}_{\mathfrak{a}_a}(\tau|_t) \leftrightarrow \mathrm{occ}_{R_a N_a L_a}(\tau|_t)\}_{\tau,t}$.

Proof. Fix a trace τ and a thread t.

The set $\mathrm{occ}_{\mathfrak{a}^\cup}(\tau|_t)$ admits a partition $\bigcup_{\mathfrak{a}_a \in \mathfrak{a}^\cup} \mathrm{occ}_{\mathfrak{a}_a}(\tau|_t)$.

Let $N_\cup = \{R_a N_a L_a \mid \exists\, (\mathfrak{a}_a, \mathrm{atom}(R_a \triangleright N_a \triangleleft L_a)) \prec \alpha\}$. By the condition *(ii)*, i.e., $N = \sum R_a N_a L_a$, and orthogonality $\mathcal{A}$, it must be that $\mathrm{occ}_N(\tau|_t)$ admits a partition $\bigcup \mathrm{occ}_{R_a N_a L_a}(\tau|_t)$.

By assumption *(iii)*, for each predecessor atomic IRSpec $\alpha_a = (\mathfrak{a}_a, \mathrm{atom}(R_a \triangleright N_a \triangleleft L_a))$ there is an order-preserving bijection $\mathrm{occ}_{\mathfrak{a}_a}(\tau|_t) \leftrightarrow \mathrm{occ}_{R_a N_a L_a}(\tau|_t)$. We can define a global bijection from the family of corresponding partitions related by componentwise order-preserving bijections, and this concludes the proof. □

Lemma A.3 (Atom Soundness). *Let* $\mathcal{A}$ *be a collection of IRSpecs for a concurrent object* O*. For every* atomic *specification* $\alpha \in \mathcal{A}$*, the rule* Atom *is sound.*

Proof. Let $\alpha = (\mathfrak{a}, \mathrm{atom}(R \triangleright N \triangleleft L))$ be an atomic IRSpec. By definition , its corresponding canonical expression $r = canon(\alpha)$ is equal to $[RNL]$. Fix $T = Traces(O[N])$ a subset of traces of O where the paths in N are executed in isolation.

Define $T_0^\alpha = T$ and for any $k \geq 1$, let T_k^α denote the following restriction

$$T_k^\alpha = T \;\cap\; (\,(\bigcup_{0 \leq j \leq k-1} (\mathsf{Any}_{\neg r}(\!|r|\!))^j \mathsf{Any}_{\neg r}) \;\cup\; (\mathsf{Any}_{\neg r}(\!|r|\!))^k \mathsf{Any}\,)$$

consisting of traces that admit a factorization $\tau_0 \tau_{r_1} \tau_1 \cdots \tau_{r_j} \tau_j$ for $j \leq k$ with $\tau_{r_i} \in (\!|r|\!)$, where $(\!|r|\!) = \bigcup_{t \in \mathcal{T}} [\![RNL]\!]_t$.

The proof has two steps. We begin with showing by induction that $\forall\, k \geq 0.\ T \rightarrow T_k^\alpha$. This will let us infer that $T \rightarrow inf_{k \geq 0} T_k^\alpha$, that is there is a reduction from T to the greatest lower bound, i.e., inf of all T_k^α. Then we demonstrate that $inf_{k \geq 0} T_k^\alpha$ coincides with $Traces(O[r])$. By construction it equivalently describes $[\![\alpha]\!](T)$. Because T stands for $Traces(O[N])$, together it will give us that $O[N] \xrightarrow{\alpha} O[RNL]$.

Step 1. We show by induction on k that $T \rightarrow T_k^\alpha$ for all $k \geq 0$.

Base case $(k = 0)$ *.* Since $T = T_0^\alpha$, the base case hold trivially.

Inductive step. Assume that $T \rightarrow T_k^\alpha$. We must prove $T \rightarrow T_{k+1}^\alpha$.

Observe that by the definition of T_k^α it is the case that $T_k^\alpha \supseteq T_{k+1}^\alpha$. Furthermore, if we can establish that for any $\tau \in T_k^\alpha$ there exists $\tau' \in T_{k+1}^\alpha$ such that $\tau \leq \tau'$, then we can infer that $T_k^\alpha \rightarrow T_{k+1}^\alpha$. By transitivity of $\rightarrow$ and the induction hypothesis we can conclude that $T \rightarrow T_{k+1}^\alpha$.

Let τ be an arbitrary trace in T_k^α. We proceed by case analysis on the number of occurrences of $\mathfrak{a}$ in τ. We use $\#(\mathfrak{a}, \tau)$ to denote the function that inductively calculates such occurrences.

Case $\#(\mathfrak{a}, \tau) \leq k$. The proof is immediate by choosing $\tau' = \tau$. The condition of $\tau' \in T_{k+1}^\alpha$ holds, since for any $l \leq k$, the trace τ has the first l occurrences of paths π in RNL already arranged in canonical form, and for $(k+1)^{th}$ the condition holds vacuously.

Case $\#(\mathfrak{a}, \tau) \geq (k+1)$. We show how to derive a trace $\tau' \in T_{k+1}^\alpha$ from τ such that $\tau \leq \tau'$. Let $(\mathfrak{a})_t$ be the $(k+1)^{th}$ occurrence of $\mathfrak{a}$ in τ; the subscript t indicates the thread ID that executes it. By the existence of an bijective map between $\mathrm{occ}_{\mathfrak{a}}(\tau|_t)$ and $\mathrm{occ}_{\mathrm{RNL}}(\tau|_t)$ we know that there is a trace fragment $\pi_t \in [\![\mathrm{RNL}]\!]_t$ in the trace restriction to thread t denoted by $\tau|_t$, that corresponds to $(\mathfrak{a})_t$. We write $\pi_t = (\pi(i) : t)_{i=1}^{|\pi|}$ as a shorthand for pointwise tagging the control path π with the thread identifier t. Furthermore, π_t admits a factorization $\pi_t = (\pi_{\mathrm{R}})_t (\pi_{\mathrm{N}})_t (\pi_{\mathrm{L}})_t$, where $\pi_{\mathrm{R}} \in [\![\mathrm{R}]\!]$, $\pi_{\mathrm{N}} \in [\![\mathrm{N}]\!]$, and $\pi_{\mathrm{L}} \in [\![\mathrm{R}]\!]$.

Since $\tau \in T$ and T is the trace restriction $Traces(O[N])$, then it must be that τ admits the factorization $\tau_0 \tau_{N_1} \tau_1 \tau_{N_2} \cdots$, where for all i, $\tau_{N_i} \in [[\mathrm{N}]]_{t_i}$ for some $t_i \in \mathcal{T}$. Therefore we can infer that τ admits also the following factorization:

$$\exists\ \tau_1\ \tau_2 \in \mathbb{E}^*.\ \tau = \tau_1\ (\pi_{\mathrm{N}})_t\ \tau_2$$

such that the events in $(\pi_{\mathrm{R}})_t$ occur in τ_1 and are interleaved with actions from other threads, and similarly, the events in $(\pi_{\mathrm{L}})_t$ are interleaved with other actions in τ_2.

We instantiate the rule Atom for the thread t and we apply the premise giving us the contexts B_t and B'_t from the stable Hoare triple for the path $(\pi_{\mathrm{N}})_t \in [[\mathrm{N}]]$ executed by t:

$$\{B_t\}(\pi_N)_t\{B'_t\}$$

We can now exploit the contexts for the commutativity movers over KAT.

By the definition of the commutativity right mover, $R \triangleright_{t_j} B$, and the fact that $\pi_R \in [[R]]$, we infer that:

$$\exists\ \tau'_1 \in \mathbb{E}^*.\ \tau \leq \tau'_1(\pi_{\mathrm{R}})_t(\pi_{\mathrm{N}})_t\tau_2 \tag{2}$$

Similarly, by the definition of $B'\ {}_{t_j}\!\triangleleft L$ and the fact that $\pi_L \in [[L]]$:

$$\exists\ \tau'_1, \tau'_2 \in \mathbb{E}^*.\ \tau \leq \tau'_1(\pi_{\mathrm{R}})_t(\pi_{\mathrm{N}})_t(\pi_{\mathrm{L}})_t\tau'_2 \tag{3}$$

Take $\tau' = \tau'_1(\pi_{\mathrm{R}})_t(\pi_{\mathrm{N}})_t(\pi_{\mathrm{L}})_t\tau'_2$.

The final step of our induction step if to show that $\tau' \in T^{\alpha}_{k+1}$.

First, by the fact that the bijections $\forall\ t \in \mathcal{T}.\mathrm{occ}_{\mathfrak{a}}(\tau|_t) \leftrightarrow \mathrm{occ}_{\mathrm{RNL}}(\tau|_t)$ are also order-preserving, we infer that the first k occurrences of paths $\pi \in [[\mathrm{RNL}]]$ must occur in the prefix τ_1 of τ. That it, τ admitted the factorization

$$\tau = \tau_{10}(\pi_1)_{t_1}\tau_{11}(\pi_2)_{t_2}\tau_{12}\cdots(\pi_k)_{t_k}\tau_{1k}(\pi_N)_t\tau_2,\ \text{where}\ \forall\ j \leq k.\pi_j \in [[RNL]]$$

We proceed by case analysis on whether there exists $j \leq k$, s.t. $t_j = t$.

Case ($\nexists\ j \leq k.\ t_j = t$). Then all previous $(\pi_{RNL})_{t_j}$ occur in the prefix τ'_1 of τ', since their actions were not selected during the previous rewritings of constructing the witness τ'. This concludes this case

Case $\exists; j \leq k.\ t_j = t.$. We prove by contradiction that the trace fragment $(\pi_j)_{t_j}$ in τ_1 has not been affected by the previous rewritings. Let a_t be the first action encountered when traversing $(\pi_R)_t$ in reverse that occurs in $(\pi_{RNL})_{t_j}$. This implies that suffix of (π_j) and prefix of $(\pi_R\pi_N\pi_L)$ overlap. However this contradicts the non-self-overlapping condition *non-self-ovl*(RNL) of the rule.

This proves that τ' admits a factorization:

$$\tau' = \tau'_{10}(\pi_1)_{t_1}\tau'_{11}(\pi_2)_{t_2}\tau'_{12}\cdots(\pi_k)_{t_k}\tau'_{1k}(\pi_R\pi_N\pi_L)_t\tau'_2$$

where for all $j \leq k$ it is the case that $\pi_j \in [[RNL]]$ and $(\pi_R\pi_N\pi_L) \in [[RNL]]$. This concludes the proof that $\tau' \in T^{\alpha}_{k+1}$.

Step 2. We prove that $inf_{k\geq 0}T^{\alpha}_k = Traces(O[RNL]) = [[\alpha]](T)$.

Consider the sequence $(T^{\alpha}_k)_{k\geq 0}$. Since we have established in Step 1. that $T^{\alpha}_k \rightarrow T^{\alpha}_{k+1}$ for all k, this sequence forms an descending chain $T = T^{\alpha}_0 \sqsupseteq T^{\alpha}_1 \sqsupseteq T^{\alpha}_2 \ldots$. The limit of this chain is the restriction $T \cap (\mathsf{Any}_{\neg r}[[r]])^*\mathsf{Any}_{\neg r}$, which coincides with $T[r]$. Additionally, since the limit has representatives of equivalence classes that are $\preceq_r$-maximal, it is also equal to $[[\alpha]](T)$. Consequently, $Traces(O[r]) = [[\alpha]](T)$.

This concludes our proof. □

Lemma A.4 (ASeq Soundness). *Let $\mathcal{A}$ be a collection of IRSpecs for a concurrent object O. For every two* atomic *IRSpec, $\alpha_1, \alpha_2 \in \mathcal{A}$, the rule ASeq is sound*

Proof. We fix two atomic IRSpecs $\alpha_1 = (\mathfrak{a}_1, \mathrm{atom}(R_1 \triangleright N_1 \triangleleft L_1)$ and $\alpha_2 = (\mathfrak{a}_2, \mathrm{atom}(R_2 \triangleright N_2 \triangleleft L_2)$. Let $P_1 \triangleq R_1 N_1 L_1$ and $P_2 \triangleq R_2 N_2 L_2$. We show that

$$[N_1 + N_2] \overset{\alpha_1;\alpha_2}{\sqsupseteq} [P_1 + P_2].$$

Let $\tau \in Traces(O[N_1 + N_2])$. We reason by an arbitrary path in $[N_1 + N_2]$ occurring in τ that is executed in isolation by some thread. By the semantics of choice, the anchor selected by $[N_1 + N_2]$ is either an occurrence of N_1 or an occurrence of N_2.

Case 1: The selected anchor is an occurrence of N_1. By the premise $[N_1] \overset{\alpha_1}{\sqsupseteq} [P_1]$, the trace τ reduces, via α_1, to a trace τ' in $Traces(O[P_1])$.

Case 2: The selected anchor is an occurrence of N_2. By the premise $[N_2] \overset{\alpha_2}{\sqsupseteq} [P_2]$, the trace τ reduces, via α_2, to a trace τ' in $Traces(O[P_2])$.

In both cases, it remains to show that the reduction is preserved in the presence of the other pattern. This follows from the orthogonality assumption

$$P_1 \text{ orthog } P_2,$$

which ensures that occurrences of P_1 and P_2 neither overlap nor factor through one another. Hence, applying the reduction associated with one IRSpec does not create, destroy, or partially consume occurrences governed by the other.

Therefore, in either case, the resulting trace τ' belongs to

$$Traces(O[P_1 + P_2]).$$

Since τ and the path in $[N_1 + N_2]$ were arbitrary, we conclude that

$$[N_1 + N_2] \overset{\alpha_1;\alpha_2}{\sqsupseteq} [P_1 + P_2].$$

□

Lemma A.5 (Coop Soundness). *Let $\mathcal{A}$ be a collection of IRSpecs for a concurrent object O. For every* cooperative *IRSpec $\alpha \in \mathcal{A}$, the rule Coop is sound.*

Proof. Fix a cooperative IRSpec $\alpha = (\mathfrak{a}^{\cup}, \mathrm{coop}(R \triangleright N \triangleleft L))$. As in the previous proof we fix a subset of traces $T = Traces(O[N])$ where the paths in N are executed in isolation. We need to prove that the conditions of the rule soundly prove a reduction $T \longrightarrow Traces(O[r])$, where $r = \langle\langle R[N]L \rangle\rangle$ is the canonical expression corresponding to α.

We again introduce the restriction T_k^{α}, where $T_0^{\alpha} = T$ and for $k \geq 1$ it is defined as follows:

$$T_k^{\alpha} = T \ \cap \ \Big(\Big(\bigcup_{0 \leq j \leq k-1} (\mathsf{Any}_{\neg r} (\!| r |\!))^j \mathsf{Any}_{\neg r} \Big) \ \cup \ (\mathsf{Any}_{\neg r} (\!| r |\!))^k \mathsf{Any} \Big)$$

Recall that the trace semantics of an cooperative canonical expression $r = \langle\langle R[N]L \rangle\rangle$ is given by:

$$(\!| \langle\langle R[N]L \rangle\rangle |\!) = \bigcup_{t \in \mathcal{T}} \bigcup_{J \subseteq \mathcal{T} \setminus \{t\}} (\overrightarrow{\Pi}_{t_j \in J} [[\mathrm{R}]]_{t_j}) [[\mathrm{N}]]_t (\overleftarrow{\Pi}_{t_j \in J} [[\mathrm{L}]]_{t_j})$$

The proof has two parts, following a similar strategy as seen previously in the proof of Rule Atom. The first part shows the induction on k that $\forall\ k \geq 0.\ T \longrightarrow T_k^{\alpha}$. The key difference from the previous proof of Rule Atom is the interplay between the thread executing the actions in N and the other threads executing the paths represented by R and L. To this point, at each inductive step, we must show that it exists a subset of threads running the paths in R and L and which interfere with the

thread executing a control path in N. It turns out that we can define it by scanning the trace interval between two subsequence occurrences of N-paths. The second part is as before to show that the greatest lower bound of restrictions T_k^α coincides with $Traces(O[\langle\langle R[N]L\rangle\rangle])$, and equivalently, to $[[\alpha]](T)$, therefore concluding that $O[N] \to O[\langle\langle R[N]L\rangle\rangle]$.

Step 1. We show by induction on k that $T \to T_k^\alpha$ for all $k \geq 0$.

Base case ($k = 0$). Since $T = T_0^\alpha$, the base case hold trivially.

Inductive step. Assume that $T \to T_k^\alpha$. We must prove $T \to T_{k+1}^\alpha$. By definition of T_k^α we get that $T_k^\alpha \supseteq T_{k+1}^\alpha$. Next we prove that $\forall\ \tau \in T_k^\alpha.\ \exists\ \tau' \in T_{k+1}^\alpha.\ \tau \leq \tau'$.

Let τ be an arbitrary trace in T_k^α.

We use the same notation for the number of occurrences of the induction action $\mathfrak{a}^\cup$ in a trace, only this time, since the induction component $\mathfrak{a}^\cup$ is a set of actions, the helper function $\#(\mathfrak{a}^\cup, \tau)$ scans for any $\mathfrak{a} \in \mathfrak{a}^\cup$ in a trace restriction. We proceed by case analysis on $\#(\mathfrak{a}^\cup, \tau)$.

Case $\#(\mathfrak{a}^\cup, \tau) = k$. The proof is immediate by taking $\tau' = \tau$.

Case $\#(\mathfrak{a}^\cup, \tau) \geq (k+1)$. We show next how to derive $\tau' \in T_{k+1}^\alpha$ such that $\tau \leq \tau'$. We begin by identifying the k-th and $(k+1)$-th occurrences of an (possibly different) action in $\mathfrak{a}^\cup$ in τ, and then using the bijective maps f_{τ,t_k} and f_{τ,t_i}, where t_k and t_i are the threads executing the k-th and the $(k+1)$-th occurrences of an (possibly different) action in $\mathfrak{a}^\cup$, respectively, to obtain the following factorization of τ:

$$\tau = \tau_1 (\pi_{N_k})_{t_k} \tau_2 (\pi_{N_i})_{t_i} \tau_3$$

where $(\pi_{N_k})_{t_k}$, $(\pi_{N_i})_{t_i}$ are the paths represented by N corresponding to $(\mathfrak{a}')_{t_k}$, and $(\mathfrak{a}'')_{t_i}$, respectively, with $\mathfrak{a}', \mathfrak{a}'' \in \mathfrak{a}^\cup$.

We introduce some additional notation. For any trace $\tau = e_0 e_1 \ldots e_n$, we write $t[i]$ for the event at index i, $\tau[i : j]$ for the trace fragment $e_i e_{i+1} \ldots e_{j-1}$, $\tau[: i]$ for the prefix $e_0 \ldots e_{i-1}$, and $\tau[j :]$ for the suffix $e_j \ldots e_n$. We also use the following auxiliary functions. Given a regular expression k over an alphabet Σ, we define $first(\mathrm{k}) \subseteq \Sigma$ and $last(\mathrm{k}) \subseteq \Sigma$ to be the sets of symbols that can appear, respectively, as first and last symbol of any word in the language of k. For any trace τ, let $suf(\tau) \subseteq Ev^*$ (resp. $pref(\tau) \subseteq Ev^*$) denote the set of all prefix (resp. suffix) subtraces of τ.

Let k_1, k_2, i_1, i_2 be indices such that $\tau[k_1 : k_2] = (\pi_{N_k})_{t_k}$ and $\tau[i_1 : i_2] = (\pi_{N_i})_{t_i}$. It follows from the factorization of τ and that empty word is not in N that $k_1 < k_2 < i_1 < i_2$.

Next, we define the following set of thread identifiers:

$$J = \left\{ t_j \;\middle|\; \begin{array}{l} \exists\ m \in \{k_2, \ldots, i_1 - 1\}. \\ \qquad \tau[m] \in (\{t_j\} \times last(R)) \ \wedge\ suf(\tau[: i_1]|_{t_j}) \cap [[R]]_{t_j} \neq \emptyset \\ \wedge \\ \forall\ m' \in \{k_2, \ldots, i_1 - 1\}. \\ \qquad \tau[m'] \notin (\{t_j\} \times first(L)) \\ \wedge \\ pref(\tau[i_2 :]|_{t_j}) \cap [[L]]_{t_j} \neq \emptyset \end{array} \right\}$$

and show that it characterizes precisely the threads contributing to the $(k+1)$-th cooperative block of events represented by $\mathrm{coop}\{R, N, L\}$.

A thread t_j satisfies the defining property of J if it executes a path π which can be decomposed as $\pi = \pi_R \pi_L$, where $(\pi_R)_{t_j}$ is executed before $(\pi_{N_j})_{t_i}$ and $(\pi_L)_{t_j}$ is executed after. Moreover $\pi_R \in [[R]]$ and $\pi_L \in [[L]]$. We define the existence of such a path split by the presence of an occurrence, within τ_R, of the last event of some path in R, and by the absence, within τ_L, of any occurrence of a first

action of a path in L. Note that this condition excludes paths π where the last action in π_R coincides with the first action in π_L, since such a split would be ambiguous.

We prove that for every $t_j \in J$, there exists a trace τ' with the following factorization:

$$\tau' = \tau'_1 (\pi_{N_k})_{t_k} \tau'_2 (\pi_R)_{t_j} (\pi_N)_{t_i} (\pi_L)_{t_j} \tau'_3$$

such that $\tau \leq \tau'$, where $(\pi_R)_{t_j} \in [\![R]\!]_{t_j}$ and $(\pi_L)_{t_j} \in [\![L]\!]_{t_j}$.

Let t_j be an arbitrary thread identifier in J.

We proceed by instantiating the Rule Coop for the threads t_i and t_j.

By the defining property of t_j, there exists a path $\pi = \pi_R \pi_L$ executed by thread t_j such that all events in $(\pi_R)_{t_j}$ are interleaved with actions of other threads and executed before $(\pi_{N_i})_{t_i}$, while all events of $(\pi_L)_{t_j}$ are executed after $(\pi_{N_i})_{t_i}$ and are also interleaved with other threads' actions. Moreover $\pi_R \in [\![R]\!]$ and $\pi_L \in [\![L]\!]$.

According to the rule premises, the contexts $C_{t_j} = \mathrm{sp}_{t_j}(\mathrm{R}, true)$ and $C'_{t_j} = \mathrm{wp}_{t_j}(\mathrm{L}, true)$ summarize the effect of the surrounding paths executed by thread t_j. Namely, C_{t_j} characterizes the states reachable after executing a path in R, while C'_{t_j} characterizes the states from which a path in L can be executed.

It must be that C_{t_j} holds for the state immediately after executing the last action in $(\pi_R)_{t_j}$, and C'_{t_j} holds for the state before the first action in $(\pi_L)_{t_j}$

Consider now the last action, say c, in π_R, and the first action, say d, in π_L, both executed by t_j. This will give us the factorization:

$$\tau = \tau_1 (\pi_{N_k})_{t_k} \tau_{21} (c_{t_j}) \tau_{22} (\pi_N)_{t_i} \tau_{31} (d_{t_j}) \tau_{32}$$

Observe that any actions in τ_{22} are executed by threads in the environment, different from both t_i and t_j. Furthermore, no action in $bk(N)$ can occur in the τ_{22}. Indeed, suppose for contradiction, that there exists such an action occurring in τ_{22}. Since traces in $Traces(O[N])$ execute N-paths in isolation, and $bk(N) \overset{t_i}{\leftrightarrow} N$, it must be that any action in $bk(N)$ can arise only from such paths. Therefore, such action would have to belong to another isolated occurrence of an N-path. By the bijection $\mathfrak{a}^{\cup} \leftrightarrow N$, there would be an additional occurrence of an action from $\mathfrak{a}^{\cup}$ strictly between the k-th and the $(k+1)$-th ones, contradicting the choice of $(\pi_{N_i})_{t_i}$ as the $(k+1)$-th occurrence.

Hence, τ_{22} is free of synchronization primitives from $bk(N)$. By the premise $IStable(C_{t_j}, \overline{bk(N)})$, the context C_{t_j} is therefore preserved through this interval, that is C_{t_j} holds before executing the path $(\pi_N)_{t_i}$.

Then, the Hoare-style premise $\{C_{t_j}\} N_{t_i} \{C'_{t_j}\}$ states that execution of the anchor (π_N) by thread t_i transforms the context C_{t_j} to the one from which the fallback path π_L is enabled for thread t_j.

We can now exploit the contexts for the commutativity movers over KAT. By the definition of the commutativity right mover, $R \triangleright_{t_j} C$, and the fact that $\pi_R \in [\![R]\!]$, we infer that:

$$\exists\ \tau'_1, \tau'_2 \in Ev^*.\ \tau \leq \tau'_1 (\pi_{N_k})_{t_k} \tau'_2 (\pi_R)_{t_j} (\pi_N)_{t_i} \tau_3 \tag{4}$$

Similarly, by the definition of $C'_{t_j} \triangleleft L$ and the fact that $\pi_L \in [\![L]\!]$:

$$\exists\ \tau'_1, \tau'_2, \tau'_3 \in Ev^*.\ \tau \leq \tau'_1 (\pi_{N_k})_{t_k} \tau'_2 (\pi_R)_{t_j} (\pi_N)_{t_i} (\pi_L)_{t_j} \tau'_3 \tag{5}$$

Therefore we must have:

$$\forall\ t_j \in J.\ \exists\ \tau'_1, \tau'_2, \tau'_3 \in Ev^*.\ \tau \leq \tau'_1 (\pi_{N_k})_{t_k} \tau'_2 (\pi_R)_{t_j} (\pi_N)_{t_i} (\pi_L)_{t_j} \tau'_3$$

Assume that $J = \{t_{j_1}, \ldots, t_{j_m}\}$ such that $t_{j_1} < \ldots < t_{j_m}$. The condition $R \rightleftarrows R$ (resp. $L \rightleftarrows L$) states that every path in R (resp. L) consists of commutative actions, and consequently, we can rearrange them in increasing (resp. decreasing) order of their thread identifiers, it follows that:

$$\tau \leq \tau'_1 (\pi_{N_k})_{t_k} \tau'_2 (\pi_{R_1})_{t_{j_1}} \ldots (\pi_{R_m})_{t_{j_m}} (\pi_N)_{t_i} (\pi_{L_m})_{t_{j_m}} \ldots (\pi_{k_1})_{t_{j_1}} \tau'_3$$

for some $\tau'_1, \tau'_2, \tau'_3$. Let τ' denote this reordered trace. For brevity of notation, we write:

$$\tau' = \tau'_1 (\pi_{N_k})_{t_k} \tau'_2 \overrightarrow{(\pi_{R_J})}_{t_J} (\pi_{N_i})_{t_i} \overleftarrow{(\pi_{L_J})}_{t_J} \tau'_3$$

The last step of this inductive step is to show that $\tau' \in T^{\alpha}_{k+1}$.

By the fact that the bijections $\forall\ t \in \mathcal{T}.\mathrm{occ}_{\mathfrak{a}\cup}(\tau|_t) \leftrightarrow \mathrm{occ}_{\mathrm{N}}(\tau|_t)$ are also order-preserving, we infer that the first k occurrences of trace fragments $\pi \in [[N]]$ must occur in the prefix of τ up to the occurrence of $(\pi_{N_i})_{t_i}$. That it, τ admitted the factorization

$$\tau = \tau_{10}\ \tau_1\ \tau_{11}\ \tau_2\ \tau_{12}\ \cdots\ (\tau_k)\ \tau_{1k}\ (\pi_{N_i})_{t_i}\ \tau_3$$

where for any $l \leq k$ there exists t_l and $J_l \subseteq \mathcal{T}\backslash\{t_l\}$ such that:

$$\tau_l \in (\overrightarrow{\Pi}_{t_j \in J_l} [[\mathrm{R}]]_{t_j})\, [[\mathrm{N}]]_{t_l}\, (\overleftarrow{\Pi}_{t_j \in J_l} [[\mathrm{L}]]_{t_j})$$

We show next that the first k canonical groups of events have not been affected by the construction of τ'. The proof is by case analysis on whether there exists $l \leq k$, s.t. $J_l \cap J \neq \emptyset$. That is, whether there exists at least one thread t_j that contributes to both an early τ_l and also to the cooperative block corresponding to $(k+1)$-th occurrence of $\mathfrak{a}$.

Case $\nexists\ l \leq k.\ J_l \cap J \neq \emptyset$. Then all trace fragments (τ_l) occur in the prefix $\tau'_1 (\pi_{N_k})_{t_k} \tau'_2$ of τ', since their actions were not actively involved during the rewritings that constructed τ'. This concludes this case.

Case $\exists\ l \leq k.\ J_l \cap J \neq \emptyset$. Let t_ξ be some thread that is in both J and J_l. We prove by contradiction that the trace fragment τ_l in τ has not been affected by the construction. We focus on the restriction of the trace τ' to t_ξ. Suppose it is affected, and let a be the first action in the path $\pi_{R_a}\ \pi_{L_a}$ within the earlier τ_l that occurs also in the path $\pi_{R_b}\ \pi_{L_b}$ contributing to the $(k+1)$-th cooperative block. This implies that the suffix $\pi_{R_a}\ \pi_{L_a}$ and prefix of $\pi_{R_b}\ \pi_{L_b}$ overlap. However this contradicts the non-ambiguity condition of α.

It follows that τ' admits a factorization:

$$\tau' = \tau_{10}\ \tau_1\ \tau_{11}\ \tau_2\ \tau_{12}\ \cdots\ (\tau_k)\ \tau_{1k}\ (\pi_{N_i})_{t_i}\ \tau'_2 \overrightarrow{(\pi_{R_J})}_{t_J} (\pi_{N_i})_{t_i} \overleftarrow{(\pi_{L_J})}_{t_J} \tau'_3$$

where for all $l \leq k$ it is the case that $\tau_l \in (|\langle\langle R[N]L\rangle\rangle|)$. This concludes the proof that $\tau' \in T^{\alpha}_{k+1}$.

Step 2. We prove that $inf_{k\geq 0} T^{\alpha}_k = Traces(O[\langle\langle R[N]L\rangle\rangle] = [[\alpha]](T)$. The argument is analogous to that in the soundness proof of the Atom, mut. mut. The infimum $inf_{k\geq 0} T^{\alpha}_k$ is given by the limit of the chain $T = T^{\alpha}_0 \sqsupseteq T^{\alpha}_1 \rightarrow T^{\alpha}_2 \ldots$, which is precisely $Trace(O[\langle\langle R[N]L\rangle\rangle]$, and since the chain keeps the representatives of equivalent classes that are also $\preceq_r$-maximal, it follows that it also coincides with $[[\alpha]](T)$.

This concludes our proof. □

Lemma A.6 (CStruct Soundness). *Let $\mathcal{A}$ be a collection of IRSpecs for a concurrent object O. For every* cooperative *IRSpec $\alpha \in \mathcal{A}$, the rule CStruct is sound.*

Proof. By Lemma 5.2

$$\langle\langle R[\textstyle\sum_{N\in N_\cup} N]L\rangle\rangle = \textstyle\sum_{N\in N_\cup} \langle\langle R[N]L\rangle\rangle.$$

Let $\tau \in Traces(O[\sum_{N\in N_\cup} N])$. Then τ is centered around some $N \in N_\cup$.

By the premise $[N] \overset{\alpha}{\sqsupseteq} \langle\langle R[N]L\rangle\rangle$, τ reduces to a trace in $Traces(O[\langle\langle R[N]L\rangle\rangle])$, hence to a trace in

$$Traces\Big(O\Big[\sum_{N\in N_\cup} \langle\langle R[N]L\rangle\rangle\Big]\Big) = Traces\Big(O\big[\langle\langle R[\sum_{N\in N_\cup} N]L\rangle\rangle\big]\Big).$$

Therefore the conclusion follows. □

Lemma A.7 (CSeq Soundness). *Let $\mathcal{A}$ be a collection of IRSpecs for a concurrent object O. For every two* cooperative *IRSpec, $\alpha_1, \alpha_2 \in \mathcal{A}$, the rule CSeq is sound*

Proof. Let $\tau \in Traces(O[N])$. By the premises,

$$[N] \overset{\alpha_1}{\sqsupseteq} \langle\!\langle R_1[N]L_1 \rangle\!\rangle \qquad \text{and} \qquad [N] \overset{\alpha_2}{\sqsupseteq} \langle\!\langle R_2[N]L_2 \rangle\!\rangle,$$

the trace τ can be reduced, respectively, to traces in

$$Traces(O[\langle\!\langle R_1[N]L_1 \rangle\!\rangle]) \quad \text{and} \quad Traces(O[\langle\!\langle R_2[N]L_2 \rangle\!\rangle]).$$

By orthogonality of R_1L_1 and R_2L_2, the two reductions are non-interfering: the surrounding paths selected by one rule neither overlap with nor factor through those selected by the other. Hence both reductions may be soundly combined around the same anchor N.

Further, since $R_1 \rightleftarrows R_2$ and $L_1 \rightleftarrows L_2$, the right surrounding paths commute with one another and the left surrounding paths commute with one another. Therefore the set of traces obtained by applying both reductions is exactly captured by allowing, independently on each side of the anchor, a choice between the corresponding components. That is, the resulting traces belong to

$$Traces(O[\langle\!\langle (R_1 + R_2)[N](L_1 + L_2) \rangle\!\rangle]).$$

Thus every trace in $Traces(O[N])$ reduces, via $\alpha_1; \alpha_2$, to a trace in the above language, which proves

$$[N] \overset{\alpha_1;\alpha_2}{\sqsupseteq} \langle\!\langle (R_1 + R_2)[N](L_1 + L_2) \rangle\!\rangle.$$

□

Theorem A.8 (Closed form of the canonical traces). *Let O be a concurrent object, and let $\mathcal{A}$ be a collection of IRSpecs that is covering and orthogonal for O. If $\prec_1$ is an admissible linear extension of the precedence order on $\mathcal{A}$, then $O_{can}^{\prec_1} \sqsubseteq O$ and $(r_1 + \cdots + r_m)^*$ is an abstraction of the $O_{can}^{\prec_1}$, that is:*

$$Traces(O_{can}^{\prec_1}) \subseteq (r_1 + \cdots + r_m)^*,$$

where each r_i is a basic reduction derived from the rules.

Proof. Let $\prec_1 \triangleq \alpha_1; \cdots; \alpha_n$ be an admissible linear extension of the precedence order on $\mathcal{A}$, and let

$$O \overset{\alpha_1}{\sqsupseteq} O_1 \overset{\alpha_2}{\sqsupseteq} \cdots \overset{\alpha_n}{\sqsupseteq} O_{\text{can}}^{\prec_1}$$

be the corresponding canonicalization sequence.

Because $\prec_1$ is an admissible linear extension of the precedence order, every cooperative IRSpec is applied only after the atomic IRSpecs on which its anchor depends. Thus the sequential canonicalization is well-defined, and each factor in the above decomposition is reduced to the canonical fragment determined by its IRSpec. The second clause of covering guarantees in particular that every cooperative anchor is a sum of canonical fragments coming from preceding atomic IRSpecs, so cooperative reductions are always performed over already-canonicalized atomic anchors.

Consider an arbitrary trace $\tau \in O_{can}^{\prec_1}$. It must be that τ admits a factorization:

$$\tau = u_0\tau_1u_1\tau_2u_2\cdots$$

alternating trace fragments $\tau_i \in (\!|r|\!)$ in the language of canonical trance expressions $r = canon(\alpha)$ of some $\alpha \in \mathcal{A}$ with residual trace fragments u_i that are not grouped. We call *maximal canonical expressions.* the r's that survive the reductions sequence.

Since $O_{can}^{\prec_1} \sqsubseteq O$, τ arises from a trace of O by a sequence of sound reductions induced by the IRSpecs in A. It must be that r can take either the shape $[RNL]$ or $\langle\!\langle (\sum R_i)[\sum N](\sum L_i) \rangle\!\rangle$. By the soundness of rule CStruct, we can further simplify the second shape to $\langle\!\langle (\sum R_i)[N](\sum L_i)$, where each R_iL_i be the reduction pattern of some cooperative $\alpha in \mathcal{A}$.

We proceed in two step. First, we prove that there is no overlap between trace fragments isolated by r's. Second, we show that there are no gaps between any two subsequent trace fragments in the language of r's, that is, all u_i are empty.

1.Canonical fragments do not overlap. We prove that if τ_i and τ_j are two trace fragments isolated by maximal canonical expressions, then they are disjoint.

Suppose for contradiction that two such fragments overlap.

If both fragments are isolated by atomic expressions $R_iN_iL_i$ and $R_jN_jL_j$, then a nonempty overlap induces either:

(i) a nonempty suffix of $R_iN_iL_i$ that is also a prefix of $R_jN_jL_j$, or vice versa, or
(ii) one reduction pattern occurs as a factor of the other.

Both possibilities contradict orthogonality. If $i = j$, then the overlap is a self-overlap of the same reduction pattern, which is ruled out by the non-self-overlap premise used in the soundness of the rule Atom

If one or both fragments are isolated by cooperative expressions, project the overlap to the thread(s) participating in the corresponding canonical expression. Any nonempty overlap must then occur either inside the anchor component, between a surrounding component and an anchor, or between surrounding components on the same side of the anchor. In every case, the overlap induces either a prefix/suffix overlap or a factorization relation between the underlying reduction patterns. Again this contradicts orthogonality. Therefore no two maximal canonical fragments overlap. Hence the family of maximal canonical fragments in τ is pairwise disjoint.

2. There are no gaps between consecutive canonical fragments. We prove that every residual fragment u_j is empty.

Assume, for contradiction, that some u_j contains an event a_t that does not belong to any maximal canonical fragment of τ. Let $\mathrm{k_{op}}$ be the implementation of the operation in which a occurs. Since $\mathcal{A}$ is covering for O, condition (i) of covering implies that every control path of $\mathrm{k_{op}}$ is contained in the starred union of IRSpec reduction patterns. Hence there exists some $\alpha \in \mathcal{A}$ such that a occurs in the reduction pattern $[\alpha]$. Since the event a_t occurs in the trace, the corresponding occurrence of $[\alpha]$ must also occur in the underlying execution. But $\prec_1$ enforces every α-reduction during the canonicalization sequence, and by soundness that occurrence must therefore be grouped into the canonical fragment $canon(\alpha)$ in τ. This contradicts the assumption that a_t lies outside all maximal canonical fragments. Therefore every u_j is empty.

Combining the two steps, we conclude that τ admits a factorization

$$\tau = \tau_1\tau_2 \cdots \tau_\ell$$

such that each τ_k belongs to the language of a basic canonical expression r_i for some $i \in \{1, \ldots, m\}$. Equivalently,

$$\tau \in (r_1 + \cdots + r_m)^*.$$

Since τ was arbitrary, it follows that

$$Traces(O_{can}^{\prec_1}) \subseteq (r_1 + \cdots + r_m)^*.$$

This proves the theorem.

□

B Concurrent object implementations

B.1 Counter

Decrement:

```
WhileStar(
   (c:=ctr)_{S};
   OR(
       [[cas(c=ctr;ctr--)/true]]_{D}
       (exit)_{Y};
  )+(
   [[cas(c≠ctr)/false]]_{G}
  )
)
```

Increment:

```
WhileStar(
   (c:=ctr)_{R};
   OR(
       [[cas(c=ctr;ctr++)/true]]_{I}
       (exit)_{X};
  )+(
   [[cas(c≠ctr)/false]]_{F}
  )
)
```

B.2 Michael-Scott Queue

Enqueue:

```
WhileStar(
   (node_t *node=)_{a};
     (tail=Q.tail)_{b};
     (next=tail→next)_{c};
   If([Q.tail==tail]_{d}) {
   If([next==null]_{e}) {
     If([CAS(&tail→next,next,node)/true]_{f}) {
       (return 1)_{g};
     }
     { assume([CAS(&tail→next,next,node)/false]_{h});
       (skip)_{s};
}
   }
   { assume([next≠null]_{j});
     (skip)_{s};
}
  }
  { assume([Q.tail≠tail]_{k});
   (skip)_{s};
}
)
```

Dequeue:

```
WhileStar(
  (int pval;head=Q.head;tail=Q.tail;next=head→next;)_{A};
  If([Q.head==head]_{B}) {
    If([head==tail]_{C}) {
      If([next==null]_{D}) {
        (return 0)_{E};
      }
      { assume([next≠null]_{F});
        (skip)_{G};
    }
  }
  { assume([head≠tail]_{H});
          (pval=next→val)_{I};
    If([CAS(&Q→head,head,next)/true]_{J}) {
      (ret pval)_{K};
    }
    { assume([CAS(&Q→head,head,next)/false]_{L});
      (skip)_{M}; }
  }
 }
 { assume([Q.head≠head]_{N}); (skip)_{O}; }
)
```

Tail advancer:

```
WhileStar(
  (tail=Q.tail;next=tail→next;)_{Q};
  If([next≠null]_{R}) {
  If([CAS(&Q→tail,tail,next)/true]_{S}) {
    (ret 0)_{T};
  }
  { assume([CAS(&Q→tail,tail,next)/false]_{U}); (skip)_{V}; }
 }
 { assume([next==null]_{W}); (skip)_{X};}
)
```

B.3 Treiber's Stack

Push:

```
WhileStar(
  (n=malloc)_{a};
  (n→val=item)_{b};
  (oldTop=top)_{c};
  (n→next=oldTop)_{d};
  OR(
      [[cas(top=oldTop,oldTop:=n)/true]]_{e}
      (exit)_{g};
  )+(
    [[cas(top≠oldTop)/false]]_{f}
  )
)
```

Pop:

```
WhileStar(
  (oldTop=top)_{h};
  If([oldTop==NULL]_{i}) {
    (ret0)_{j};
  }
  { assume([oldTop≠NULL]_{k});
    (newtop=oldTop→next)_{l};
      OR(
          [[cas(top=oldTop,oldTop:=n)/true]]_{m}
          (ret(val))_{n};
      )+(
          [[cas(top≠oldTop)/false]]_{o}
      )
  }
)
```

B.4 Hendler et al. Elimination Stack

Push:

```
WhileStar(
    (n=malloc)_{a};
    (n→val=item)_{b};
    (oldTop=top)_{c};
    (n→next=oldTop)_{d};
  OR(
      [[cas(top=oldTop,oldTop:=n)/true]]_{e}
      (exit)_{g};
  )+(
      [[cas(top≠oldTop)/false]]_{f}
  )

  (location[mytid] = p; pos = nondet();)_{A};
  (him = collision[pos])_{B};

  WhileARW([[CAS(&collision[pos], him, mytid)/false]]_{C}){
    (him = collision[pos])_{B};
  } [[CAS(&collision[pos], him, mytid)/true]]_{D};

  If([him ≠NULL]_{E}) {
    (q = location[him])_{F};
  Switch{
      Case([q≠NULL]_{r}/\[q.id=him]_{s}/\[p.op≠q.op]_{t}) {
      If([CAS(&location[mytid],p,NULL)/true]_{H}) {
          If([CAS(&location[him], q, p)/true]_{I}) {
              (return p)_{J};
          }
          { assume([CAS(&location[him], q, p)/false]_{K});
              (skip (continue))_{L}; }
      }
      { assume([CAS(&location[mytid],p,NULL)/false]_{M});
              (val = NULL)_{N};
              (location[mytid] = NULL)_{O};
```

```
            (return val)_{P};
      }
      }
      Case([q≠NULL]_{r}/\[q.id=him]_{s}/\[p.op=q.op]_{w}) {skip_{R};}
      Case([q≠NULL]_{r}/\[q.id≠him]_{v}/\[p.op≠q.op]_{t}) {skip_{R};}
      Case([q≠NULL]_{r}/\[q.id≠him]_{v}/\[p.op=q.op]_{w}) {skip_{R};}
      Case([q=NULL]_{u}/\[q.id=him]_{s}/\[p.op≠q.op]_{t}) {skip_{R};}
      Case([q=NULL]_{u}/\[q.id=him]_{s}/\[p.op=q.op]_{w}) {skip_{R};}
      Case([q=NULL]_{u}/\[q.id≠him]_{v}/\[p.op≠q.op]_{t}) {skip_{R};}
      Case([q=NULL]_{u}/\[q.id≠him]_{v}/\[p.op=q.op]_{w}) {skip_{R};}
   }
 }
 { assume([him == NULL]_{S});
   (skip)_{T}; }
   If([CAS(&location[mytid],p,NULL)/false]_{U}) {
      (val = NULL)_{V};
      (location[mytid] = NULL)_{W};
      (return val)_{X};
 }
 { assume([CAS(&location[mytid],p,NULL)/true]_{Y});
   (skip)_{Z};
 }
)
```

Pop:

```
WhileStar(
  (oldTop=top)_{h};
  If([oldTop==NULL]_{i}) {
    (ret0)_{j};
  }
  { assume([oldTop≠NULL]_{k});
      (newtop=oldTop→next)_{l};
      OR(
         [[cas(top=oldTop,oldTop:=n)/true]]_{m}
         (ret(val))_{n};
      )+(
         [[cas(top≠oldTop)/false]]_{o}
      )
   }
   (location[mytid] = p; pos = nondet();)_{A};
   (him = collision[pos])_{B};
   WhileARW([[CAS(&collision[pos], him, mytid)/false]]_{C}){
     (him = collision[pos])_{B};
   } [[CAS(&collision[pos], him, mytid)/true]]_{D};

     If([him ≠NULL]_{E}) {
       (q = location[him])_{F};
       Switch{
          Case([q≠NULL]_{r}/\[q.id=him]_{s}/\[p.op≠q.op]_{t}) {
             If([CAS(&location[mytid],p,NULL)/true]_{H}) {
                If([CAS(&location[him], q, NULL)/true]_{I}) {
                (return NULL)_{J};
```

```
                }
                { assume([CAS(&location[him], q, NULL)/false]_{K});
                (skip (continue))_{L};
                }
            }
            { assume([CAS(&location[mytid],p,NULL)/false]_{M});
                (val = location[mytid].input)_{N};
                (location[mytid] = NULL)_{O};
                (return val)_{P};}
        }
        Case([q≠NULL]_{r}/\[q.id=him]_{s}/\[p.op=q.op]_{w}) {skip_{R};}
        Case([q≠NULL]_{r}/\[q.id≠him]_{v}/\[p.op≠q.op]_{t}) {skip_{R};}
        Case([q≠NULL]_{r}/\[q.id≠him]_{v}/\[p.op=q.op]_{w}) {skip_{R};}
        Case([q=NULL]_{u}/\[q.id=him]_{s}/\[p.op≠q.op]_{t}) {skip_{R};}
        Case([q=NULL]_{u}/\[q.id=him]_{s}/\[p.op=q.op]_{w}) {skip_{R};}
        Case([q=NULL]_{u}/\[q.id≠him]_{v}/\[p.op≠q.op]_{t}) {skip_{R};}
        Case([q=NULL]_{u}/\[q.id≠him]_{v}/\[p.op=q.op]_{w}) {skip_{R};}
  }

}
{ assume([him == NULL]_{S});(skip)_{T}; }
 If([CAS(&location[mytid],p,NULL)/false]_{U}) {
     (val = location[mytid].data)_{V};
     (location[mytid] = NULL)_{W};
     (return val)_{X};
}
{ assume([CAS(&location[mytid],p,NULL)/true]_{Y});
  (skip)_{Z}; }
)
```

B.5 RDCSS

```
WhileStar(
  (init)_{S};
       OR(
  (r=CAS(d→DATA_ADDR,d→exp_data,d)/true)_{A};)
 +((r=CAS(d→DATA_ADDR,d→exp_data,d)/false)_{F};
 )

  If([isDescriptor(r)]_{b}) {
  If([*r→CONTROL_ADDR == r→exp_control]_{c}) {
    OR( (CAS(r→DATA_ADDR, r, r→new_data)/true)_{d};)
      +((CAS(r→DATA_ADDR, r, r→new_data)/false)_{u};)
  }
  { assume([*r→CONTROL_ADDR ≠r→exp_control]_{e});
    OR( (CAS(r→DATA_ADDR, r, r→exp_data)/true)_{f};)
     +( (CAS(r→DATA_ADDR, r, r→exp_data)/false)_{v}; )
}
}
{ assume([!isDescriptor(r)]_{g});(skip)_{h}; }

  While([isDescriptor(r)]_{i}){
```

```
      OR(
    (r=CAS(d→DATA_ADDR,d→exp_data,d)/true)_{A};)
  +((r=CAS(d→DATA_ADDR,d→exp_data,d)/false)_{F}; )

    If([isDescriptor(r)]_{b}) {
      If([*r→CONTROL_ADDR == r→exp_control]_{c}) {
        OR(
          (CAS(r→DATA_ADDR, r, r→new_data)/true)_{d};)
        +((CAS(r→DATA_ADDR, r, r→new_data)/false)_{u}; )
      }
      { assume([*r→CONTROL_ADDR ≠r→exp_control]_{e});
        OR(
          (CAS(r→DATA_ADDR, r, r→exp_data)/true)_{f};)
        +((CAS(r→DATA_ADDR, r, r→exp_data)/false)_{v};)
      }
  }{ assume([!isDescriptor(r)]_{g}); (skip)_{h}; }

  }
assume([not isDescriptor(r)]_{j});

If([r == d→exp_data]_{k}) {
  If([*d→CONTROL_ADDR == d→exp_control]_{l}) {
    OR(
      (CAS(d→DATA_ADDR, d, d→exp_data)/true)_{m};)
     +((CAS(d→DATA_ADDR, d, d→exp_data)/false)_{s};)
  }
  { assume([*d→CONTROL_ADDR ≠d→exp_control]_{n});
    OR( (CAS(d→DATA_ADDR, d, d→exp_data)/true)_{o};)
     +( (CAS(d→DATA_ADDR, d, d→exp_data)/false)_{t}; )
  }
}
assume([r ≠d→exp_data]_{p});(skip)_{q};
(return r)_{r};
)
```

C IRSpecs verified by QRef

Michael-Scott Queue IRSpecs

$\alpha_{\text{fAEnqS}} = (a_i : \texttt{f}, \quad \texttt{atom}\{\texttt{abcde} \triangleright \texttt{f} \triangleleft \texttt{g}\})$
Enqueue happy path

$\alpha_{\text{fCEnqF}} = (a_i : \texttt{f}, \quad \texttt{coop}\{(\texttt{(abcde)+(abcd)}) \triangleright \texttt{abcdefg} \triangleleft \texttt{(h+j)s}\})$
Two related paths: fail the CAS, or fail the next==null test

$\alpha_{\text{fADeqS}} = (a_i : \texttt{J}, \quad \texttt{atom}\{\texttt{ABCHI} \triangleright \texttt{J} \triangleleft \texttt{K}\})$
Dequeue happy path

$\alpha_{\text{fCDeqF}} = (a_i : \texttt{J}, \quad \texttt{coop}\{(\texttt{(ABCHI)+(AN)}) \triangleright \texttt{ABCHIJK} \triangleleft \texttt{((LM)+(O))}\})$
Two related paths:
(i) Related deq path that fails the CAS:
(ii) Also deq path that finds Q.head!=head

$\alpha_{\text{fAEmp}} = (a_i : \texttt{D}, \quad \texttt{atom}\{\texttt{ABC} \triangleright \texttt{D} \triangleleft \texttt{E}\})$
Read-only path when queue is empty

$\alpha_{\text{fAOne}} = (a_i : \texttt{F}, \quad \texttt{atom}\{\texttt{ABC} \triangleright \texttt{F} \triangleleft \texttt{G}\})$
Read-only path when queue has 1 element, but need advancer to act

$\alpha_{\text{fAAdvS}} = (a_i : \texttt{S}, \quad \texttt{atom}\{\texttt{QR} \triangleright \texttt{S} \triangleleft \texttt{T}\})$
Advancer

$\alpha_{\text{fCAdvF}} = (a_i : \texttt{S}, \quad \texttt{coop}\{(\texttt{(QR)+(abc)}) \triangleright \texttt{QRST} \triangleleft \texttt{((UV)+(ks))}\})$
Two related paths:
(i) Related path that fails the CAS
(ii) Also a path in Enq that fails due to advancer

$\alpha_{\text{fAAdv}} = (a_i : \texttt{W}, \quad \texttt{atom}\{\texttt{Q} \triangleright \texttt{W} \triangleleft \texttt{X}\})$
Advancer read-only path when nothing to do

with order: $\alpha_{\text{fAEnqS}} \prec \alpha_{\text{fCEnqF}}, \alpha_{\text{fADeqS}} \prec \alpha_{\text{fCDeqF}}, \alpha_{\text{fAAdvS}} \prec \alpha_{\text{fCAdvF}}$

Harris et al. RDCSS IRSpecs

α_{dataS} = $(a_i\!:\!\mathtt{A},\quad \mathtt{atom}\{\mathtt{(S+i)} \triangleright \mathtt{A} \triangleleft\})$
Successful modification in the first data CAS.

α_{dataF} = $(a_i\!:\!\mathtt{A},\quad \mathtt{coop}\{\mathtt{(S+(i))} \triangleright \mathtt{SA} \triangleleft \mathtt{F}\})$
- causing others to fail their CAS

α_{controlS} = $(a_i\!:\!\mathtt{m},\quad \mathtt{atom}\{\mathtt{jkl} \triangleright \mathtt{m} \triangleleft \mathtt{r}\})$
Main thread next succeeds the second control CAS

α_{controlF} = $(a_i\!:\!\mathtt{m},\quad \mathtt{coop}\{\mathtt{bc} \triangleright \mathtt{jklmr} \triangleleft \mathtt{u}\})$
- helpers fail their control CAS

α_{helpS} = $(a_i\!:\!\mathtt{d},\quad \mathtt{atom}\{\mathtt{bc} \triangleright \mathtt{d} \triangleleft\})$
A helper thread fails to cas, but then succeeds in helping

α_{helpF} = $(a_i\!:\!\mathtt{d},\quad \mathtt{coop}\{\mathtt{((bc)+(jkl)+j+g)} \triangleright \mathtt{bcd} \triangleleft \mathtt{(u+(sr)+(pqr)+h)}\})$
- other helpers thread fails to cas, and then fail to help
- also impacts main thread's if() after the loop
- also impacts main thread's attempt to CAS
- also impacts other helpers who will find that !isDescriptor()

α_{restoreS} = $(a_i\!:\!\mathtt{o},\quad \mathtt{atom}\{\mathtt{jkn} \triangleright \mathtt{o} \triangleleft \mathtt{r}\})$
Main thread finds bad control and succeds in restoring

α_{restoreF} = $(a_i\!:\!\mathtt{o},\quad \mathtt{coop}\{\mathtt{be} \triangleright \mathtt{jknor} \triangleleft \mathtt{v}\})$
- causes helpers to fail in helping restore

$\alpha_{\text{helperRestoreS}}$ = $(a_i\!:\!\mathtt{f},\quad \mathtt{atom}\{\mathtt{be} \triangleright \mathtt{f} \triangleleft\})$
Helper thread does the restoring

$\alpha_{\text{helperRestoreF}}$ = $(a_i\!:\!\mathtt{f},\quad \mathtt{coop}\{\mathtt{((jkn)+(j))} \triangleright \mathtt{bef} \triangleleft \mathtt{((tr)+(pqr))}\})$
- causing the main thread to noop on the CAS
- also impacts main thread's if() after the loop

with order: $\alpha_{\text{dataS}} \prec \alpha_{\text{dataF}}, \alpha_{\text{controlS}} \prec \alpha_{\text{controlF}}, \alpha_{\text{helpS}} \prec \alpha_{\text{helpF}}, \alpha_{\text{restoreS}} \prec \alpha_{\text{restoreF}}, \alpha_{\text{helperRestoreS}} \prec \alpha_{\text{helperRestoreF}}$

Counter IRSpecs

$$\alpha_{\text{fARIX}} = (a_i\!:\!\texttt{I},\quad \text{atom}\{\texttt{R} \triangleright \texttt{I} \triangleleft \texttt{X}\})$$
Successful increment CAS

$$\alpha_{\text{fASDY}} = (a_i\!:\!\texttt{D},\quad \text{atom}\{\texttt{S} \triangleright \texttt{D} \triangleleft \texttt{Y}\})$$
Successful decrement CAS

$$\alpha_{\text{fCrix}} = (a_i\!:\!\texttt{I+D},\quad \text{coop}\{\texttt{(R)} \triangleright \texttt{(RIX+SDY)} \triangleleft \texttt{(F)}\})$$
Successful incr/decr causes others to fail their incr CAS

$$\alpha_{\text{fCsdy}} = (a_i\!:\!\texttt{I+D},\quad \text{coop}\{\texttt{(S)} \triangleright \texttt{(RIX+SDY)} \triangleleft \texttt{(G)}\})$$
Successful incr/decr causes others to fail their decr CAS

with order: $\alpha_{\text{fCrix}} \prec \alpha_{\text{fARIX}}, \alpha_{\text{fCrix}} \prec \alpha_{\text{fASDY}}, \alpha_{\text{fCsdy}} \prec \alpha_{\text{fARIX}}, \alpha_{\text{fCsdy}} \prec \alpha_{\text{fASDY}}$

Treiber's Stack IRSpecs

$$\alpha_{\text{fAPushS}} = (a_i\!:\!\texttt{e},\quad \text{atom}\{\texttt{abcd} \triangleright \texttt{e} \triangleleft \texttt{g}\})$$
A push succeeds

$$\alpha_{\text{fAPopS}} = (a_i\!:\!\texttt{m},\quad \text{atom}\{\texttt{hkl} \triangleright \texttt{m} \triangleleft \texttt{n}\})$$
A pop succeeds

$$\alpha_{\text{fCPushF}} = (a_i\!:\!\texttt{e+m},\quad \text{coop}\{\texttt{(abcd)} \triangleright \texttt{((abcdeg)+(hklmn))} \triangleleft \texttt{(f)}\})$$
A successful push/pop causes other pushes to fail

$$\alpha_{\text{fCPopF}} = (a_i\!:\!\texttt{e+m},\quad \text{coop}\{\texttt{(hkl)} \triangleright \texttt{((abcdeg)+(hklmn))} \triangleleft \texttt{(o)}\})$$
A successful push/pop causes other pops to fail

$$\alpha_{\text{fAPopEmp}} = (a_i\!:\!\texttt{j},\quad \text{atom}\{\triangleright \texttt{h} \triangleleft \texttt{ij}\})$$
An attempted pop on an empty stack

with order: $\alpha_{\text{fCPushF}} \prec \alpha_{\text{fAPushS}}, \alpha_{\text{fCPopF}} \prec \alpha_{\text{fAPopS}}, \alpha_{\text{fCPushF}} \prec \alpha_{\text{fAPopS}}, \alpha_{\text{fCPopF}} \prec \alpha_{\text{fAPushS}}$

Hendler Elimination Stack IRSpecs

α_{fAPushS} = (a_i:e, atom{abcd ▹ e ◃ g})
A push succeeds

α_{fAPopS} = (a_i:m, atom{hkl ▹ m ◃ n})
A pop succeeds

α_{fCPushF} = (a_i:e+m, coop{(abcd) ▹ ((abcdeg)+(hklmn)) ◃ (f)})
A successful push/pop causes other pushes to fail

α_{fCPopF} = (a_i:e+m, coop{(hkl) ▹ ((abcdeg)+(hklmn)) ◃ (o)})
A successful push/pop causes other pops to fail

α_{fAPopEmp} = (a_i:j, atom{ ▹ h ◃ ij})
An attempted pop on an empty stack

α_{fApubS} = (a_i:D, atom{AB(CB)* ▹ D ◃ })
First loop
- pick a position to publish my Thread descriptor, and loop until we can replace him with me

α_{fCpubF} = (a_i:D, coop{AB(CB)* ▹ AB(CB)*D ◃ C})

α_{fAhappyS} = (a_i:I, atom{EFrstH ▹ I ◃ J})
Happy path to I: CAS(location[him], q, p/NULL) succeeds

$\alpha_{\text{fAhappyFI}}$ = (a_i:I, coop{EFrstHK ▹ EFrstHIJ ◃ L})
Happy path causes CAS competitors at to fail at I

$\alpha_{\text{fAhappyFH}}$ = (a_i:I, coop{EFrstM ▹ EFrstHIJ ◃ NOP})
Happy path causes CAS competitors at to fail at H

$\alpha_{\text{fAfallbackS}}$ = (a_i:U, atom{((ST)+(EF((rsw)+(rvt)+(rvw)+(ust)+(usw)+(uvt)+(uvw))R)) ▹ U ◃ V
catch the other cases which fail line 8 or 10 for some reason, but succeed UVWX

$\alpha_{\text{fAfallbackF}}$ = (a_i:U, coop{((ST)+(EF((rsw)+(rvt)+(rvw)+(ust)+(usw)+(uvt)+(uvw))R))Y ▹ ((S
catch the other cases which fail line 8 or 10 for some reason, but fail UVWX

with order: $\alpha_{\text{fCPushF}} \prec \alpha_{\text{fAPushS}}$, $\alpha_{\text{fCPopF}} \prec \alpha_{\text{fAPopS}}$, $\alpha_{\text{fCPushF}} \prec \alpha_{\text{fAPopS}}$, $\alpha_{\text{fCPopF}} \prec \alpha_{\text{fAPushS}}$, $\alpha_{\text{fApubS}} \prec \alpha_{\text{fCpubF}}$, $\alpha_{\text{fAhappyS}} \prec \alpha_{\text{fAhappyFI}}$, $\alpha_{\text{fAhappyS}} \prec \alpha_{\text{fAhappyFH}}$, $\alpha_{\text{fAfallbackS}} \prec \alpha_{\text{fAfallbackF}}$

D Commutativity Contexts for IRSpecs Proofs

D.1 Counter

*IRSpec α_{incS} (**atom**).* A Increment succeeds.

B_{t_i}	$c_{t_i} = ctr$
B'_{t_i}	$\exists\, oldCtr.\ ctr = oldCtr + 1 \ \wedge\ c_{t_i} = oldCtr$

*IRSpec α_{decS} (**atom**).* A Decrement succeeds.

B_{t_i}	$c_{t_i} = ctr$
B'_{t_i}	$\exists\, oldCtr.\ ctr = oldCtr - 1 \ \wedge\ c_{t_i} = oldCtr$

*IRSpec α_{incF} (**coop**).* A Increment fails due to another successful Inc or Dec.

C_{t_j}	$c_{t_j} = ctr$
C'_{t_j}	$c_{t_j} \neq ctr$

*IRSpec α_{decF} (**coop**).* A Increment fails due to another successful Inc or Dec.

C_{t_j}	$c_{t_j} = ctr$
C'_{t_j}	$c_{t_j} \neq ctr$

D.2 Treiber Stack

*IRSpec $\alpha_{fAPushS}$ (**atom**).* A push succeeds.

B_{t_i}	$top_{t_i} = S.top \ \wedge\ n_{t_i} \neq NULL \ \wedge\ n_{t_i} \neq S.top \ \wedge$ $n_{t_i} \rightarrow next = S.top \ \wedge\ n_{t_i} \rightarrow val = item_{t_i}$
B'_{t_i}	$\exists\, S.top'.\ top_{t_i} = S.top' \ \wedge\ n_{t_i} \neq NULL \ \wedge\ n_{t_i} \neq S.top' \ \wedge$ $n_{t_i} \rightarrow next = S.top' \ \wedge\ n_{t_i} \rightarrow val = item_{t_i} \ \wedge\ S.top = n_{t_i}$

*IRSpec α_{fAPopS} (**atom**).* A pop succeeds.

B_{t_i}	$top_{t_i} = S.top \ \wedge\ S.top \neq NULL \ \wedge$ $newtop_{t_i} = S.top \rightarrow next$
B'_{t_i}	$\exists\, S.top'.\ top_{t_i} = S.top' \ \wedge\ S.top' \neq NULL \ \wedge$ $newtop_{t_i} = S.top' \rightarrow next \ \wedge\ S.top = newtop_{t_i}$

*IRSpec $\alpha_{fCPushF}$ (**coop**).* A successful push or pop causes other pushes to fail

C_{t_j}	$\big(top_{t_j} = S.top \ \wedge\ n_{t_j} \neq NULL \ \wedge\ n_{t_j} \neq S.top \ \wedge$ $n_{t_j} \rightarrow next = S.top \ \wedge\ n_{t_j} \rightarrow val = item_{t_j}\big)$
C'_{t_j}	$S.top \neq top_{t_j}$

*IRSpec $\alpha_{fCPushF}$ (**coop**).* A successful push or pop causes other pops to fail

C_{t_j}	$\big(top_{t_j} = S.top \ \wedge\ S.top \neq NULL \ \wedge$ $newtop_{t_j} = S.top \rightarrow next\big)$
C'_{t_j}	$S.top \neq top_{t_j}$

IRSpec $\alpha_{fAPopEmp}$ *(* ***atom****).* An attempted pop on an empty stack

B_{t_i}	$S.top = NULL$
B'_{t_i}	$top_{t_i} = S.top \ \wedge \ S.top = NULL$

D.3 Michael-Scott Queue

IRSpec α_{fAEnqS} *(****atom****).* Enqueue happy path `abcdefg`.

B_{t_i}	$Q.tail = tail_{t_i} \ \wedge \ tail_{t_i} \rightarrow next = next_{t_i} \ \wedge \ next_{t_i} = NULL \wedge$ $node_{t_i} \neq NULL \ \wedge \ node_{t_i} \neq tail \ \wedge \ node_{t_i} \rightarrow next = NULL$
B'_{t_i}	$\exists\, Q.tail' \,.\, Q.tail' = tail_{t_i} \ \wedge \ tail_{t_i} \rightarrow next = next_{t_i} \ \wedge \ next_{t_i} = NULL \wedge$ $node_{t_i} \neq NULL \ \wedge \ node_{t_i} \neq tail_{t_i} \ \wedge \ node_{t_i} \rightarrow next = NULL \wedge$ $Q.tail \rightarrow next = node_{t_i}$

IRSpec α_{fCEnqF} *(****coop****).* Enqueuers fail for one of two reasons: `((abcde)+(abcd))` ... `(h+j)s`.

C_{t_j}	$(Q.tail = tail_{t_j} \ \wedge \ tail_{t_j} \rightarrow next = next_{t_j} \ \wedge \ next_{t_j} = NULL)$ $\vee (Q.tail = tail_{t_j} \ \wedge \ tail_{t_j} \rightarrow next = next_{t_j})$
C'_{t_j}	$(tail_{t_j} \rightarrow next \neq next_{t_j}) \vee (next_{t_j} \neq null)$

IRSpec α_{fADeqS} *(****atom****).* Dequeue happy path `ABHIJK`.

B_{t_i}	$Q.head = head_{t_i} \ \wedge \ pval_{t_i} = next_{t_i} \rightarrow val \ \wedge \ head_{t_i} \neq tail_{t_i} \wedge$ $Q.tail = tail_{t_i} \ \wedge \ next_{t_i} = head_{t_i} \rightarrow next$
B'_{t_i}	$\exists\, Q.head' \,.\, Q.head' = head_{t_i} \ \wedge \ pval_{t_i} = next_{t_i} \rightarrow val \ \wedge \ head_{t_i} \neq tail_{t_i} \wedge$ $Q.tail = tail_{t_i} \ \wedge \ next_{t_i} = head_{t_i} \rightarrow next \wedge$ $Q.head \rightarrow next = next_{t_i}$

IRSpec α_{fCDeqF} *(****coop****).* Dequeuers fail for one of two reasons: `(ABHI+AN))` ... `(LM+O)`.

C_{t_j}	$(head_{t_j} = Q.head \wedge tail_{t_j} = Q.tail \wedge next_{t_j} = head_{t_j} \rightarrow next$ $\wedge tail_{t_j} \neq head_{t_j} pval_{t_j} = next_{t_j} \rightarrow val)$ $\vee (head_{t_j} = Q.head \wedge tail_{t_j} = Q.tail \wedge next_{t_j} = head_{t_j} \rightarrow next)$
C'_{t_j}	$Q \rightarrow head \neq head_{t_j}$

IRSpec α_{fAEmp} *(****atom****).* Dequeue attempt on empty queue `ABC,D,E`.

B_{t_i}	$Q.head = head_{t_i} \ \wedge \ pval_{t_i} = next_{t_i} \rightarrow val \ \wedge \ head_{t_i} \neq tail_{t_i}$ $\wedge\, Q.tail = tail_{t_i} \ \wedge \ next_{t_i} = head_{t_i} \rightarrow next \ \wedge \ next_{t_j} = null$
B'_{t_i}	$next_{t_j} = null$

IRSpec α_{fAOne} *(****atom****).* Dequeue attempt but advancer needed `ABC,F,G`.

B_{t_i}	$Q.head = head_{t_i} \ \wedge \ pval_{t_i} = next_{t_i} \rightarrow val \ \wedge \ head_{t_i} \neq tail_{t_i}$ $\wedge\, Q.tail = tail_{t_i} \ \wedge \ next_{t_i} = head_{t_i} \rightarrow next$
B'_{t_i}	$next_{t_j} \neq null$

IRSpec α_{fAAdvS} ***(atom)***. Advancer succeeds `QR,S,T`.

B_{t_i}	$Q.tail = tail_{t_i} \wedge next_{t_i} = tail_{t_i} \rightarrow next \wedge next_{t_i} \neq null$
B'_{t_i}	$\exists\, oldTail.\ oldTail = tail_{t_i} \wedge Q.tail = next_{t_i}$

IRSpec α_{fCAdvF} ***(coop)***. Other advancers fail `QR+abc,...,UV+ks`.

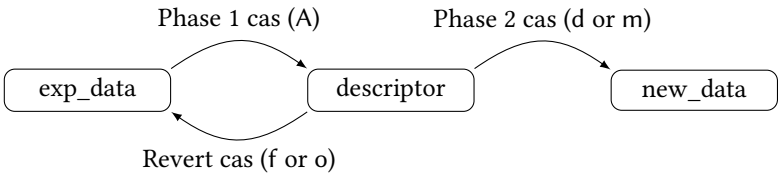
C_{t_j}	$(Q.tail = tail_{t_j} \wedge next_{t_j} = tail_{t_j} \rightarrow next \wedge next_{t_j} \neq null)$ $\vee (node_{t_j} = Q.tail \wedge next_{t_j} = node_{t_j} \rightarrow next)$
B'_{C_j}	$Q.tail \neq tail_{t_j}$

IRSpec α_{fAAdvS} ***(atom)***. Advancer with nothing to do `Q,W,X`.

B_{t_i}	$Q.tail = tail_{t_i} \wedge next_{t_i} = tail_{t_i} \rightarrow next$
B'_{t_i}	$next_{t_i} = null$

D.4 Harris et al RDCSS

This algorithm involves two registers, below notated *DATA* and *CTL*. We use *DATA* to mean the global value of the data register (which can be an expected value, or a descriptor or a new value), and *CTL* for the global control register. A successful full invocation of an RDSS write will change *DATA* from its expected value (`d->exp_data`) first to descriptor `d` ("Phase 1") and then to `d->new_data` ("Phase 2") as depicted below. After Phase 1, if *CTL* does not have the expected value, the *DATA* register will be "Reverted."



IRSpec α_{dataS}. Successful modification in Phase 1 CAS.

B_{t_i}	$DATA = \texttt{d->exp_data}_{t_i}$
B'_{t_i}	$\exists\, DATA'.DATA' = \texttt{d->exp_data}_{t_i} \ \wedge\ DATA = \texttt{d}_{t_i}$ $\wedge\ \texttt{r}_{t_i} = \texttt{d->exp_data}_{t_i}(\text{old value}) \ \wedge\ \neg isDesc(\texttt{r}_{t_i})$

IRSpec α_{dataF}. Others fail their Phase 1 CAS attempt.

C_{t_j}	$DATA = \texttt{r}_{t_j}\texttt{->exp_data} \wedge isDesc(\texttt{r}_{t_j})$
C'_{t_j}	$DATA \neq \texttt{r}_{t_j}\texttt{->exp_data}$

IRSpec $\alpha_{controlS}$. Main thread successful Phase 2 cas after the loop.

B_{t_i}	$\neg isDesc(\texttt{r}_{t_i}) \wedge \texttt{r}_{t_i} = \texttt{d}_{t_i}\texttt{->exp_data} \wedge CTL = \texttt{d}_{t_i}\texttt{->exp_control} \wedge DATA = \texttt{d}_{t_i}$
B'_{t_i}	$\exists\, DATA'.DATA' = \texttt{d}_{t_i}$ $\wedge\ \neg isDesc(\texttt{r}_{t_i}) \wedge \texttt{r}_{t_i} = \texttt{d}_{t_i}\texttt{->exp_data} \ \wedge\ CTL = \texttt{d}_{t_i}\texttt{->exp_control}$ $\wedge\ DATA = d_{t_i}\texttt{->new_data}$

IRSpec $\alpha_{controlF}$. Helpers fail their Phase 2 CAS.

C_{t_j}	$DATA = \mathtt{r}_{t_j}\texttt{->exp_data} \wedge isDesc(\mathtt{r}_{t_j}) \wedge CTL = \mathtt{r}_{t_j}\texttt{->exp_control}$
C'_{t_j}	$DATA \neq \mathtt{r}_{t_j}\texttt{->exp_data}$

IRSpec α_{helpS}. A helper t_i fails Phase 1 cas, but then succeeds in helping with Phase 2.

B_{t_i}	$isDesc(\mathtt{r}_{t_i}) \wedge CTL = \mathtt{r}_{t_i}\texttt{->exp_control}$
B'_{t_i}	$\exists\, DATA'.DATA' = \mathtt{r}_{t_i}$ $\wedge\, isDesc(\mathtt{r}_{t_i}) \wedge CTL = \mathtt{r}_{t_i}\texttt{->exp_control} \wedge DATA = \mathtt{r}_{t_i}\texttt{->new_data}$

IRSpec α_{helpF}. The above helper t_i causes other helpers to fail the Phase 2 CAS, and also cause the main thread's Phase 2CAS to fail.

C_{t_j}	$((isDesc(\mathtt{r}_{t_j}) \wedge CTL = \mathtt{r}_{t_j}\texttt{->exp_control} \wedge DATA = \mathtt{r}_{t_j})$ $\vee(DATA = \mathtt{d}_{t_j} \wedge \mathtt{r}_{t_j} = \mathtt{d}_{t_j}\texttt{->exp_data} \wedge CTL = \mathtt{d}_{t_j}\texttt{->exp_control})$ $\vee(\neg isDesc(\mathtt{r}_{t_j}))$
C'_{t_j}	$((DATA \neq \mathtt{r}_{t_j})$ $\vee(DATA \neq \mathtt{d}_{t_j})$ $\vee(DATA \neq \mathtt{r}_{t_j} \wedge \mathtt{d}_{t_j}\texttt{->exp_data} \neq \mathtt{r}_{t_j})$

IRSpec $\alpha_{restoreS}$. Main thread finds bad control and restores successfully.

B_{t_i}	$\neg isDesc(\mathtt{r}_{t_i}) \wedge \mathtt{r}_{t_i} = \mathtt{d}_{t_i}\texttt{->exp_data}$ $\wedge DATA = \mathtt{d}_{t_i} \wedge CTL \neq \mathtt{d}_{t_i}\texttt{->exp_control}$
B'_{t_i}	$\exists\, DATA'.DATA' = \mathtt{d}_{t_i}$ $\neg isDesc(\mathtt{r}_{t_i}) \wedge \mathtt{r}_{t_i} = \mathtt{d}_{t_i}\texttt{->exp_data}$ $\wedge DATA = \mathtt{d}_{t_i}\texttt{->exp_data} \wedge CTL \neq \mathtt{d}_{t_i}\texttt{->exp_control}$

IRSpec $\alpha_{restoreF}$. Main thread t_i restores successfully, causing helper t_j to fail CAS.

C_{t_j}	$isDesc(\mathtt{r}_{t_j}) \wedge CTL \neq \mathtt{r}_{t_j}\texttt{->exp_control}$
C'_{t_j}	$DATA \neq \mathtt{r}_{t_j}$

IRSpec $\alpha_{helperRestoreS}$. Helper does the restoration.

B_{t_i}	$isDesc(\mathtt{r}_{t_i}) \wedge DATA = \mathtt{r}_{t_i} \wedge CTL \neq \mathtt{r}_{t_i}\texttt{->exp_control}$
B'_{t_i}	$\exists\, DATA'.DATA' = \mathtt{r}_{t_i}$ $\wedge isDesc(\mathtt{r}_{t_i}) \wedge DATA = \mathtt{r}_{t_i}\texttt{->exp_data} \wedge CTL \neq \mathtt{r}_{t_i}\texttt{->exp_control}$

IRSpec $\alpha_{helperRestoreF}$. Causes the main thread or other helpers to fail.

C_{t_j}	$((\neg isDesc(\mathtt{r}_{t_j}) \wedge \mathtt{r}_{t_j} = \mathtt{d}_{t_j}\texttt{->exp_data} \wedge CTL \neq \mathtt{d}_{t_j}\texttt{->exp_control})$ $\vee(\neg isDesc(\mathtt{r}_{t_j})))$
C'_{t_j}	$(DATA \neq \mathtt{d}_{t_j})$ $\vee(\mathtt{r}_{t_j} \neq \mathtt{d}_{t_j}\texttt{->exp_data})$